

Winter School of AI and Robotics [REPORT] (No Comments)

| Table of Contents

Day 1 - Session 1

Python Installation

Python Basics

Task: Write a program to run Fibonacci Series

Task: Write a program to find nth element in Fibonacci Sequence.

NumPy Installation

Task: Slice a 4×4 2D array and make it 3×3, containing elements of the bottom right 9 elements.

Task: Write a program to multiply two matrices of different dimensions.

OpenCV Library

Image Creation on CV2

Drawing a Line on the created image

Drawing an Ellipse on the Created Image

Drawing a Rectangle on the Created Image

Reading a Local Image using OpenCV

Day 1 - Session 2

Task: Write a code to show the projectile motion of a ball.

Task: Write a code of a moving box in 2D, which bounces when hits the boundary of the canvas.

Task: Write a code with a ball bouncing as in task 2, with a static circle on the canvas, such that the canvas closes when the ball touches the circle.

Task: Write a code for a simple Pacman program of the ball opening and closing its mouth with keyboard input

Day 2 - Session 1

Types of Images

Color Spaces

RGB Color Spaces

HSV Color Spaces

CMYK Color Spaces

MATPLOTLIB Module

Changing of Color Spaces (Using OpenCV)

Rotating an Image using OpenCV

Rotating an Image 90 degrees without using OpenCV

Rotating an Image 180 degrees without using OpenCV

Padding using OpenCV

Threshold Value in OpenCV

Task: Download a Greyscale image of a coin and find the threshold value above which you can see the circle around the coin

Task: Take any Image and convert it from RGB to Greyscale without using OpenCV

Task: Take any Image and convert it from RGB to Black and White without using OpenCV

Camera Videos using OpenCV

Standard Code

Task: Take any short Video and Resize it. Try drawing a circle over the region of interest or any other shape.

Day 2 - Session 2

Image Segmentation

Clustering

K-Means Clustering

Scatter Plot (MATPLOTLIB) -

Scatter Plot for Random points in a 2D Array -

Task: Write a program to execute K-means clustering of the above set of points.

Task: Write a program to execute K-means clustering for given two images, used RGB coordinates.

Day 3 - Session 2

Kernels

Gaussian Blur

Bilateral Filtering -

Edge Detection-

Methods of Edge Detection -

Sobel Method -

Task: Using Sobel Method, detect the edges of any image.

Laplace Method -

Task: Using Laplace Method, detect the edges of any image.

Gradients -

Canny Edge Detection -

Task: Using Canny Edge Detection, detect edges of any image

Hough Transform -

Day 4 - Session 1

Machine Learning for Computer Vision

Optimizing parameters -

Gradient Descent Method -

Task: Write the code for linear regression using gradient descent method on the California Housing Dataset.

Task: Write a program to implement Gradient Descent on a Simple Dataset and find the best fit line

Day 4 - Session 2

Logistic Regression

Task: Write a program to implement Gradient Descent on a Simple Dataset for Logistic Regression.

Universal Approximation Theorem

Convolutional Neural Networks

Parameters and Hyperparameters

Introduction of a new Loss function -

Kernels Continued

Training Models

Gradient Descent

Day 5 - Session 1

How to Represent Graphs

Edge List -

Adjacency List -

Adjacency Matrix -

Graph Traversal -

Graph to be used for BFS, DFS and Dijkstra's Algorithm Implementation

Edge List Representation of Above Graph

Breadth First Search -

Task: Write a general code to implement BFS algorithm

Task: Implement BFS Algorithm on the given graph.

Depth First Search -

Task: Write a general code to implement DFS algorithm

Task: Implement DFS Algorithm on the given graph.

Dijkstra's Algorithm

Task: Implement Dijkstra's Algorithm on the given graph.

Day 5 - Session 2

SLAM

Task: Write the general code for A* Algorithm (When diagonal movement is not allowed)

Task: Write the general code for A* Algorithm (When diagonal movement is allowed)

Task: Take an image, convert it to black and white and detect the edges using Canny Edge Detection. Pick two points and implement A* Algorithm to find the optimal path between the points. Consider the edges to be obstacles.

Day 6 - Session 1

Perform the following tasks to implement algorithms on this given maze.

Probabilistic Roadmap (PRM)

Task: Implement PRM on the given maze.

Rapidly-exploring Random Trees (RRT)

Task: Implement RRT on the given maze.

Rapidly-exploring Random Trees Star (RRT*)

Task: Implement RRT* on the given maze.

Final Day Project

[Python Code](#)

[Arduino Code](#)

[Python to Serial Monitor Code](#)

Day 1 - Session 1

Python Installation

- Everyone who had higher version of python (3.14) had to uninstall and reinstall lower version to stay on track.
- Google classroom was created and everyone was told to join. On this classroom, the assignments etc. will be posted.

Python Basics

- Variables, operators, loops etc

Task: Write a program to run Fibonacci Series

```
i = 1
j = 1

print(i)
print(j)

for k in range(1, 6):
    m = j
    j = j + i
    i = m
    print(j)
```

- Simple Data structures were covered -
 1. Dictionaries
 2. Sets
 3. Strings (Arrays of Characters)
 4. Lists
 5. Tuples

- Functions (Syntax, meaning etc)
- Exception Handling (try, except, finally)

Task: Write a program to find nth element in Fibonacci Sequence.

```
j = int(input("Which element of the Fibonacci Sequence do you want?"))
list1 = [0 for p in range(j)]

list1[0] = 1
list1[1] = 1

for i in range(2, j):
    list1[i] = list1[i-1] + list1[i-2]

print(list1[j-1])
```

NumPy Installation

pip install numpy

import numpy as np

- Many of the numpy functions were explained.

Task: Slice a 4x4 2D array and make it 3x3, containing elements of the bottom right 9 elements.

```
import numpy as np

x = np.array([[0, 0, 0, 0],
              [0, 1, 2, 3],
              [0, 4, 5, 6],
              [0, 7, 8, 9]])

y = x[1:, 1:]
print(y)
```

Task: Write a program to multiply two matrices of different dimensions.

```

import numpy as np

x = int(input("What is the number of rows in your first matrix? "))
y = int(input("What is the number of columns in your first matrix? "))
z = int(input("What is the number of columns in your second matrix? "))

m = np.full((x, y), 0, dtype=int)
n = np.full((y, z), 0, dtype=int)
for i in range(x):
    for j in range(y):
        m[i][j] = int(input("Enter element (Matrix 1): "))

for i in range(y):
    for j in range(z):
        n[i][j] = int(input("Enter element (Matrix 2): "))

final = np.full((x, z), 0, dtype=int)

def multiply(a, b):
    for i in range(x):
        for j in range(z):
            for k in range(y):
                final[i, j] = final[i, j] + a[i, k] * b[k, j]
    print(final)

multiply(m, n)

```

OpenCV Library

pip install opencv-python

- Images are made up of pixels.
- Grayscale and RGB images.

Image Creation on CV2

- Syntax -

np.full(shape, fill_value, dtype)

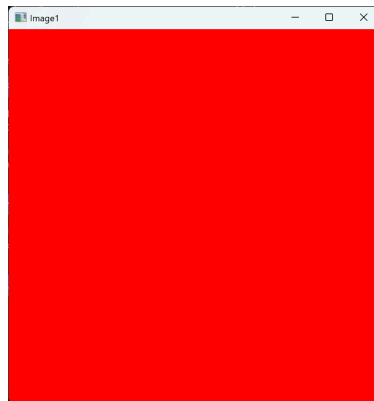
- Example-

```
img = np.full((500, 500, 0), (255, 0, 0), dtype = np.uint8)
```

```
import cv2
import numpy as np

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)

cv2.imshow("Image1", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Drawing a Line on the created image

- Syntax -

```
cv2.line(img, pt1, pt2, color, thickness)
```

- Example -

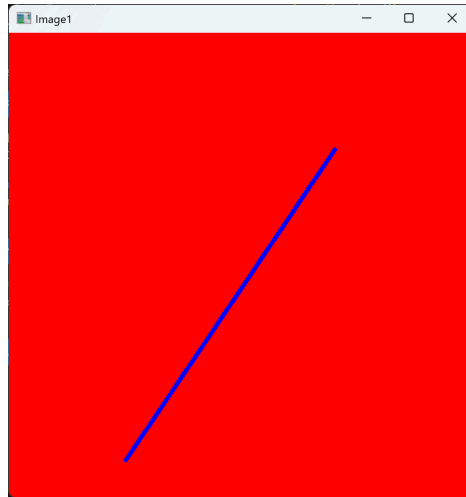
```
cv2.line(img, (350, 125), (125, 460), (255, 0, 0), 3)
```

```
import cv2
import numpy as np

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)

cv2.line(img, (350, 125), (125, 460), (255, 0, 0), 3)

cv2.imshow("Image1", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Drawing an Ellipse on the Created Image

- Syntax -

`cv2.ellipse(img, center, axes, angle, startAngle, endAngle, color, thickness)`

- Example -

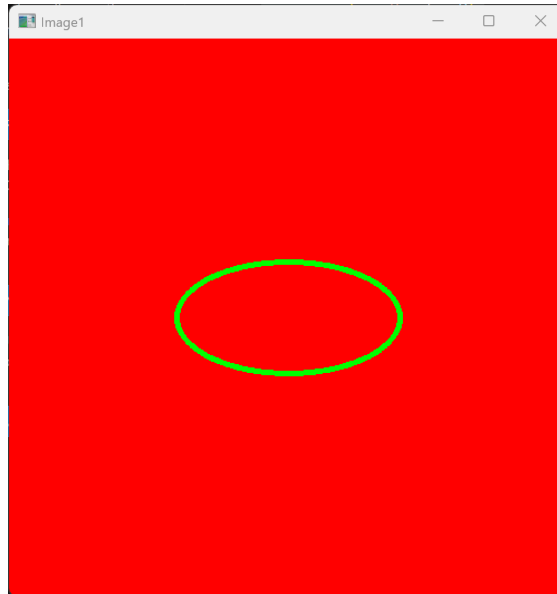
`cv2.ellipse(img, (250, 250), (100, 50), 0, 0, 360, (0, 255, 0), 3)`

```
import cv2
import numpy as np

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)

cv2.ellipse(img, (250, 250), (100, 50), 0, 0, 360, (0, 255, 0), 3)

cv2.imshow("Image1", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Drawing a Rectangle on the Created Image

- Syntax -

`cv2.rectangle(img, pt1, pt2, color, thickness)`

where pt1 is coordinate of top left corner

pt2 is coordinate of bottom right corner

- Example -

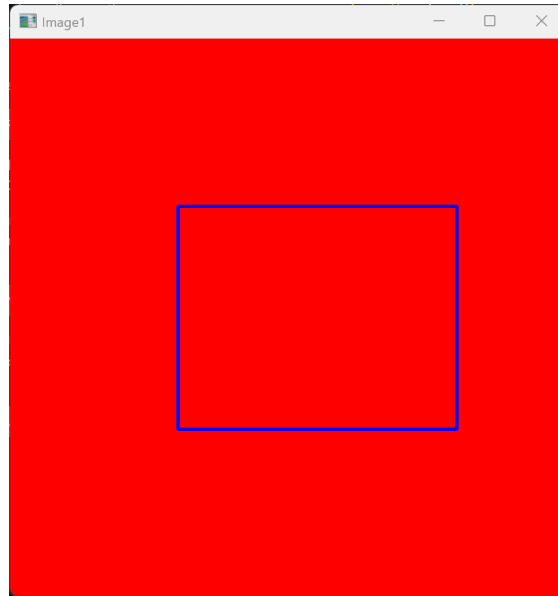
`cv2.rectangle(img, (150, 150), (400, 350), 255, 2)`

```
import cv2
import numpy as np

img = np.full((500, 500, 3), (0, 0, 255), dtype=np.uint8)

cv2.rectangle(img, (150, 150), (400, 350), 255, 2)

cv2.imshow("Image1", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Reading a Local Image using OpenCV

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.
png")

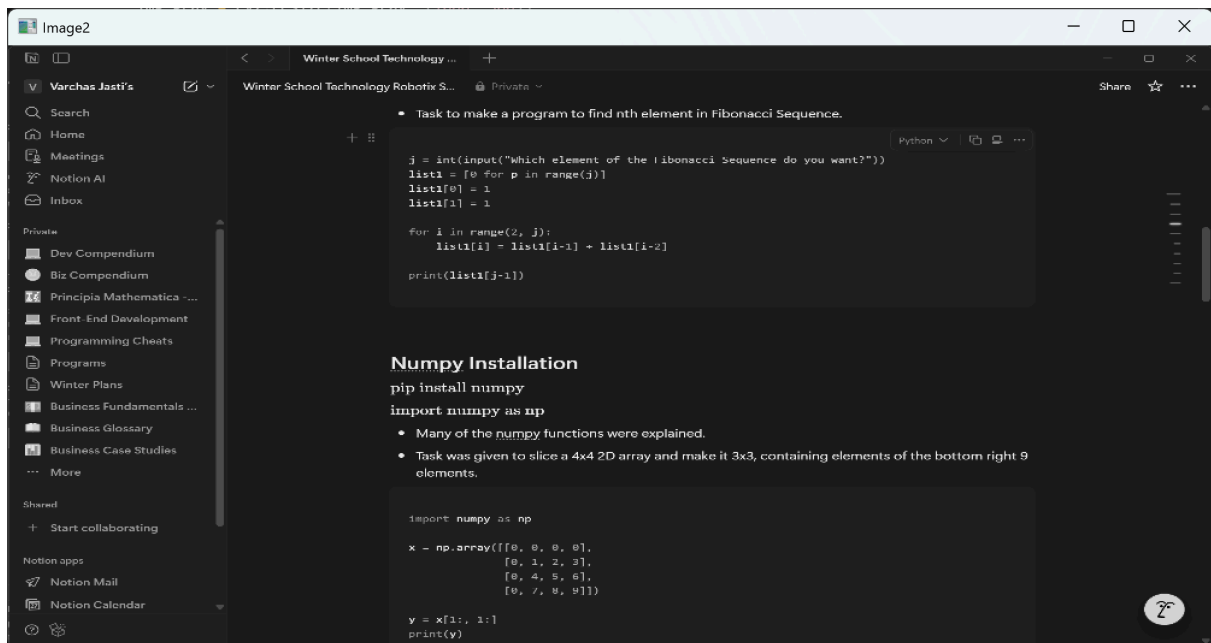
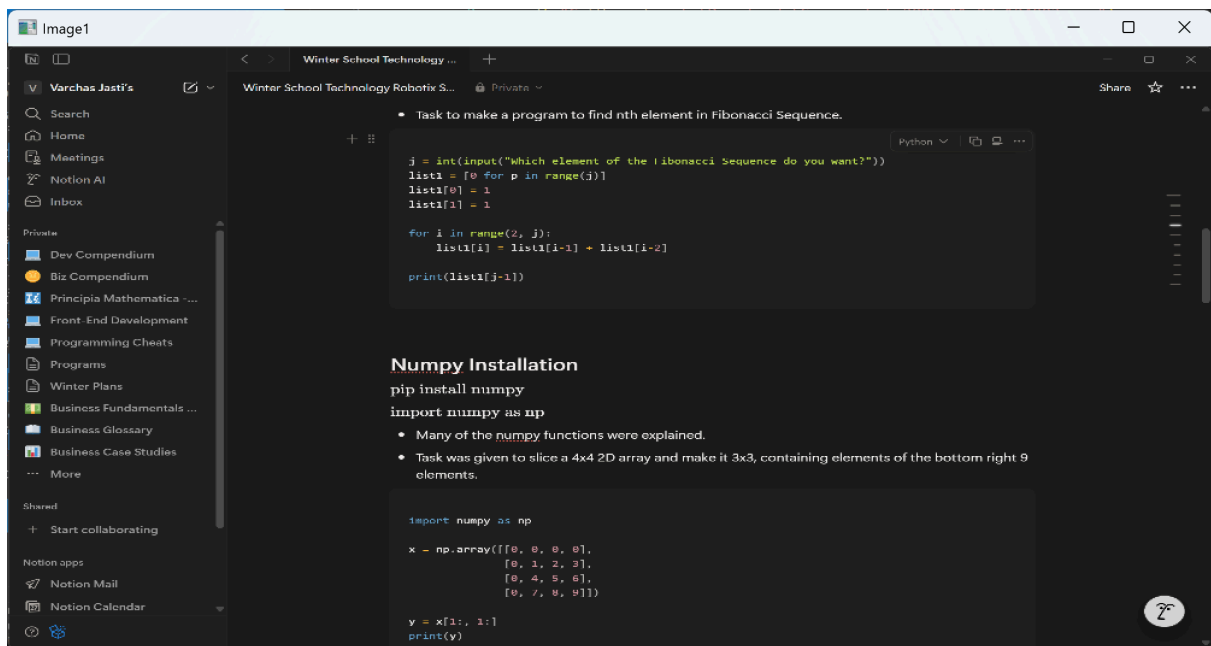
img = cv2.resize(img, (1000, 500))

img_gray = cv2.imread(
    r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png",
    cv2.IMREAD_GRAYSCALE
)

img_gray = cv2.resize(img_gray, (1000, 500))

cv2.imshow("Image1", img)
cv2.imshow("Image2", img_gray)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Day 1 - Session 2

Task: Write a code to show the projectile motion of a ball.

```
import cv2
import numpy as np
```

```

width, height = 500, 500

x, y = 0, 450
vx = 4
vy = -12
gravity = 0.4

while True:
    frame = np.full((height, width, 3), (0, 0, 255), dtype=np.uint8)

    x += vx
    vy += gravity
    y += vy

    cv2.circle(frame, (int(x), int(y)), 10, (255, 255, 255), -1)

    cv2.imshow("Simple Projectile", frame)

    if x > width or y > height:
        x, y = 0, 450
        vy = -12

    if cv2.waitKey(20) & 0xFF == ord('q'):
        break

cv2.destroyAllWindows()

```

Task: Write a code of a moving box in 2D, which bounces when hits the boundary of the canvas.

```

import cv2
import numpy as np
import keyboard as kb

i = 0
x1 = 10

```

```

y1 = 10
x2 = 50
y2 = 50
velx = 10
vely = 5

while True:
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.rectangle(img, (x1, y1), (x2, y2), 255, -1)

    cv2.imshow("Image1", img)
    cv2.waitKey(50)

    x1 += velx
    x2 += velx
    y1 += vely
    y2 += vely

    if ((x1 + x2) / 2 > 470 or (x1 + x2) / 2 < 30):
        velx = -velx

    if ((y1 + y2) / 2 > 470 or (y1 + y2) / 2 < 30):
        vely = -vely

    if (kb.is_pressed("ctrl")):
        break

cv2.destroyAllWindows()

```

Task: Write a code with a ball bouncing as in task 2, with a static circle on the canvas, such that the canvas closes when the ball touches the circle.

```

import cv2
import numpy as np
import math
import keyboard as kb

```

```

velx = 10
vely = 5

x, y = 250, 250
h = 150

while True:
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.ellipse(img, (x, y), (30, 30), 0, 0, 360, (0, 0, 255), -1)

    cv2.ellipse(img, (h, h), (30, 30), 0, 0, 360, (255, 0, 0), 2)

    cv2.imshow("Image1", img)
    cv2.waitKey(50)

    x += velx
    y += vely

    if (x > 470 or x < 30):
        velx = -velx

    if (y > 470 or y < 30):
        vely = -vely

    if (math.sqrt(((x - h) * (x - h)) + ((y - h) * (y - h))) < 60):
        break

    if (kb.is_pressed("ctrl")):
        break

cv2.destroyAllWindows()

```

Task: Write a code for a simple Pacman program of the ball opening and closing its mouth with keyboard input

```

import cv2
import numpy as np
import keyboard as kb

i = 0
j = 1

vel = 3

x, y = 25, 25

while True:
    while (kb.is_pressed("right")):
        img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

        cv2.ellipse(img, (x, y), (15, 15), 0, i, 360 - i, (255, 0, 0), -1)

        cv2.imshow("Image1", img)
        cv2.waitKey(5)

        i += j
        if (x < 482):
            x += vel

        if (i == 44 or i == -44 or i == 0):
            j = -j

    while (kb.is_pressed("down")):
        img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

        cv2.ellipse(img, (x, y), (15, 15), 90, i, 360 - i, (255, 0, 0), -1)

        cv2.imshow("Image1", img)
        cv2.waitKey(5)

        i += j
        if (y < 482):
            y += vel

```

```

    if (i == 44 or i == -44 or i == 0):
        j = -j

while (kb.is_pressed("left")):
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.ellipse(img, (x, y), (15, 15), 180, i, 360 - i, (255, 0, 0), -1)

    cv2.imshow("Image1", img)
    cv2.waitKey(5)

    i += j
    if (x > 18):
        x -= vel

    if (i == 44 or i == -44 or i == 0):
        j = -j

while (kb.is_pressed("up")):
    img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

    cv2.ellipse(img, (x, y), (15, 15), 270, i, 360 - i, (255, 0, 0), -1)

    cv2.imshow("Image1", img)
    cv2.waitKey(5)

    i += j
    if (y > 18):
        y -= vel

    if (i == 44 or i == -44 or i == 0):
        j = -j

if (kb.is_pressed("ctrl")):
    break

img = np.full((500, 500, 3), (0, 0, 0), dtype=np.uint8)

cv2.ellipse(img, (x, y), (15, 15), 0, i, 360 - i, (255, 0, 0), -1)

```

```
cv2.imshow("Image1", img)
cv2.waitKey(5)

i += j
if (i == 44 or i == -44 or i == 0):
    j = -j

cv2.destroyAllWindows()
```

Day 2 - Session 1

Types of Images

- Binary Images (Only Black and White)
- Greyscale (Black and White with concept of Intensity)
- Color Images (3 bands of monochrome colors, stores 3 bytes of data)

Color Spaces

RGB Color Spaces

- Most common.
- Additive in Nature.

HSV Color Spaces

- Hue(0-179), Saturation(0-255) and Value(0-255).
- Cylindrical representation of RGB color points.
- Attempts to depict colors as perceived by human eye.

CMYK Color Spaces

- Subtractive model. Subtracts colors red, green, blue, from white light
- White - Red = Cyan(C)
White - Green = Magenta(M)

White - Blue = Yellow(Y)

- K is color of Key Plate → Black

MATPLOTLIB Module

`pip install matplotlib`

- Used to plot graphs, charts, scattered graphs etc.

`import matplotlib.pyplot as plt`

- `plt.imshow` uses RGB color space with coordinates R, G, B coordinates only, whereas `cv2` uses B, G, R coordinates.

Changing of Color Spaces (Using OpenCV)

- Syntax

`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

- Example

`img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

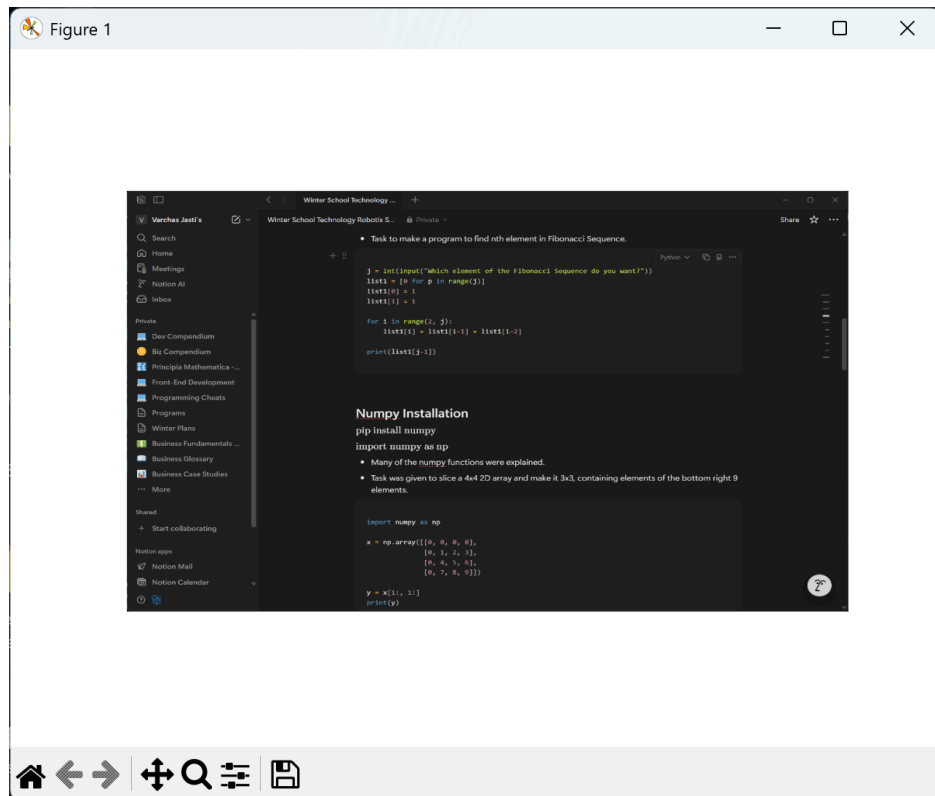
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png")

img = cv2.resize(img, (1200, 700))

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_recolor)
plt.axis('off')
plt.show()
```



Rotating an Image using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

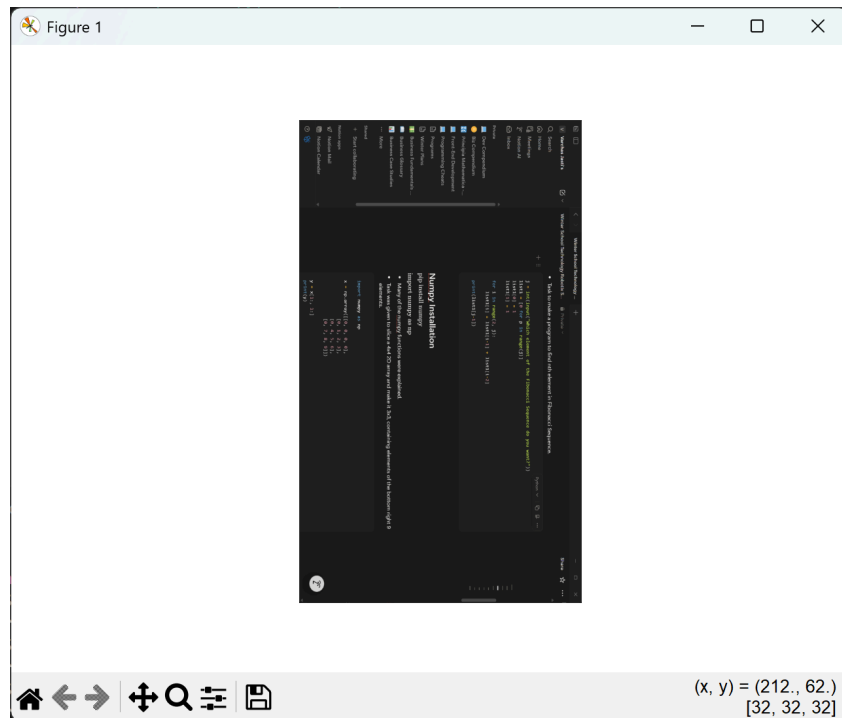
img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.png")

img = cv2.resize(img, (1200, 700))

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

img_rotated = cv2.rotate(img_recolor, cv2.ROTATE_90_CLOCKWISE)

plt.imshow(img_rotated)
plt.axis('off')
plt.show()
```



Rotating an Image 90 degrees without using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

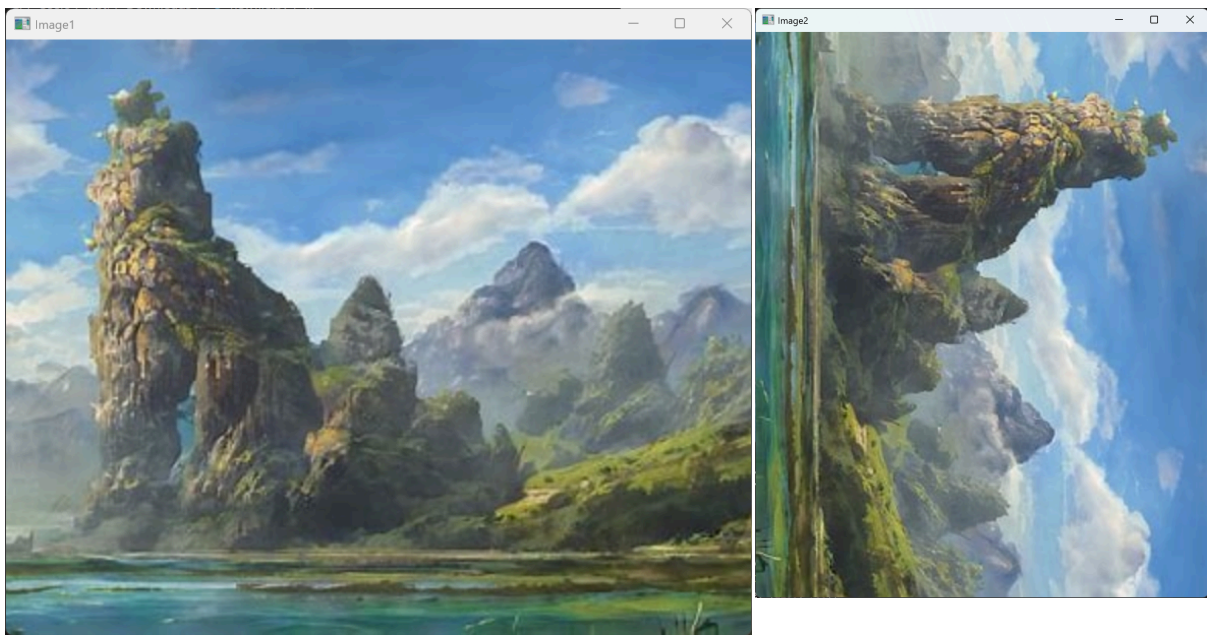
img = cv2.resize(img, (732, 586))

img_new = np.full((732, 586, 3), (0, 0, 0), dtype=np.uint8)

for i in range(732):
    for j in range(586):
        img_new[i][j] = img[585 - j][i]

cv2.imshow("Image1", img)
cv2.imshow("Image2", img_new)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Rotating an Image 180 degrees without using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

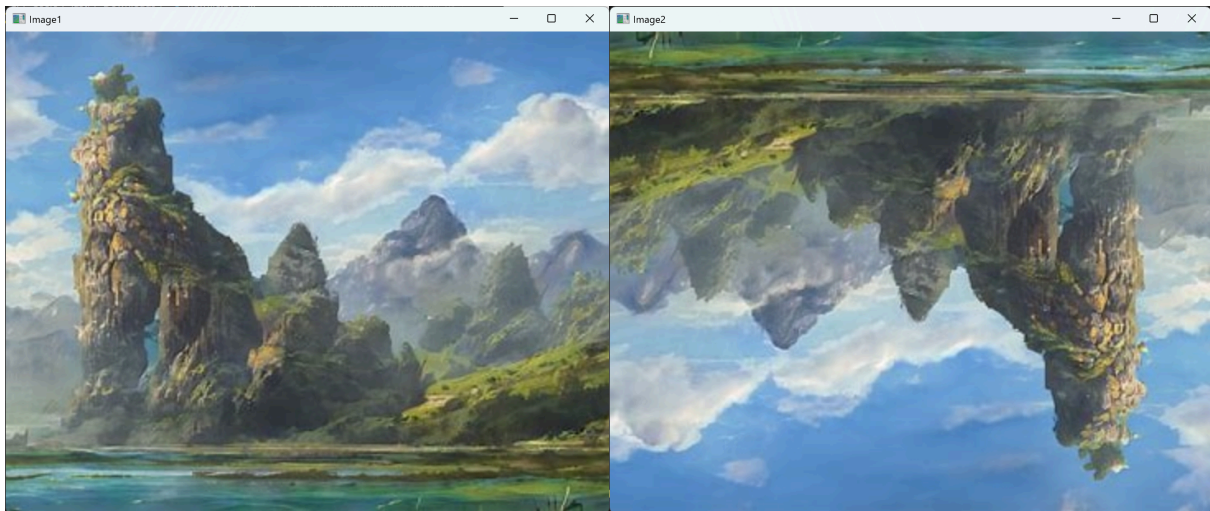
img = cv2.resize(img, (732, 586))

img_new = np.full((586, 732, 3), (0, 0, 0), dtype=np.uint8)

for i in range(732):
    for j in range(586):
        img_new[j][i] = img[585 - j][731 - i]

cv2.imshow("Image1", img)
cv2.imshow("Image2", img_new)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Padding using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

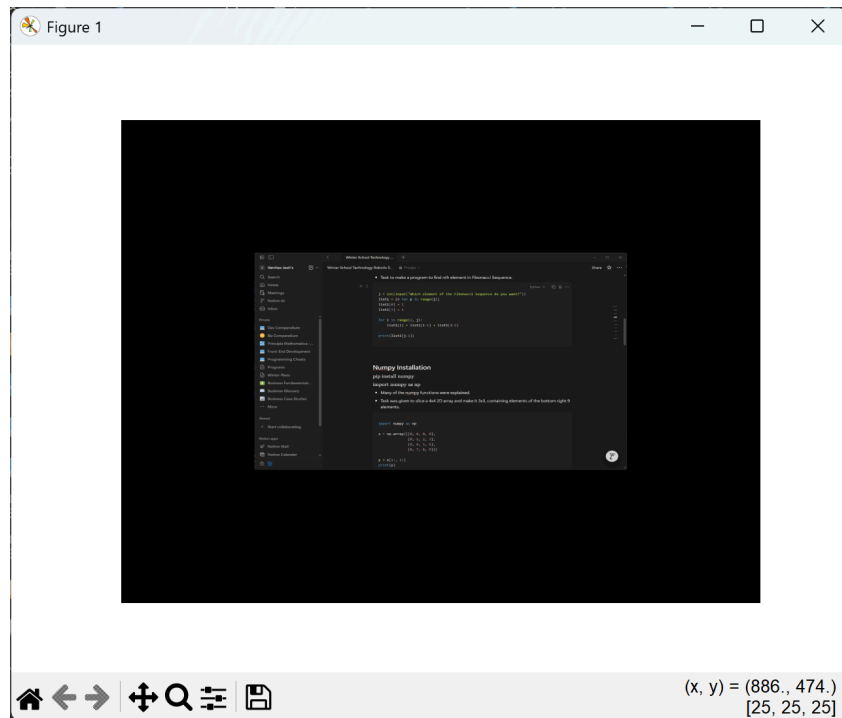
img = cv2.imread(r"C:\Users\jasti\Downloads\Screenshot 2025-11-27 124003.
png")

img = cv2.resize(img, (1200, 700))

img_recolor = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

img_padded = cv2.copyMakeBorder(
    img_recolor,
    430, 430, 430, 430,
    cv2.BORDER_CONSTANT
)

plt.imshow(img_padded)
plt.axis('off')
plt.show()
```



Threshold Value in OpenCV

Task: Download a Greyscale image of a coin and find the threshold value above which you can see the circle around the coin

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

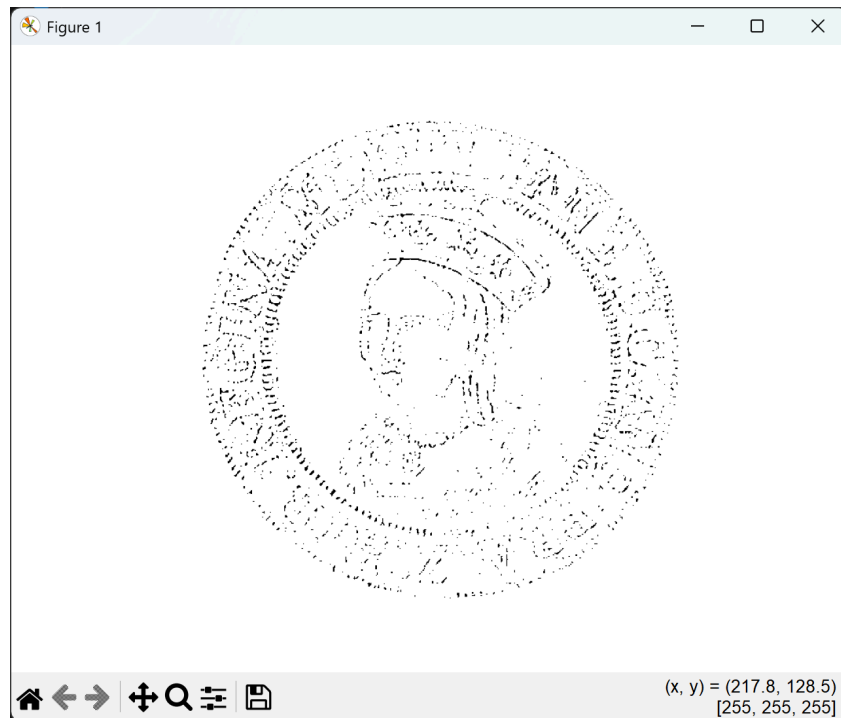
img = cv2.imread(r"C:\Users\jasti\Downloads\coin.jpg")

img = cv2.resize(img, (610, 616))

ret, binary = cv2.threshold(img, 50, 255, cv2.THRESH_BINARY)

plt.imshow(binary, cmap='gray')
plt.axis('off')
plt.show()
```

- Result: Threshold value is 50



Task: Take any Image and convert it from RGB to Greyscale without using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

img = cv2.resize(img, (732, 586))

img_new = np.full((586, 732), 0, dtype=np.uint8)

for i in range(586):
    for j in range(732):
        img_new[i][j] = (img[i][j][0] + img[i][j][1] + img[i][j][2]) / 3

plt.imshow(img_new, cmap='gray')
plt.axis('off')
plt.show()
```



Task: Take any Image and convert it from RGB to Black and White without using OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

img = cv2.resize(img, (732, 586))

img_new = np.full((586, 732), 0, dtype=np.uint8)

for i in range(586):
    for j in range(732):
        img_new[i][j] = (img[i][j][0] + img[i][j][1] + img[i][j][2]) / 3

        if (img_new[i][j] < 50):
            img_new[i][j] = 0
        else:
            img_new[i][j] = 1

plt.imshow(img_new, cmap='gray')
plt.axis('off')
plt.show()

```



Camera Videos using OpenCV

Standard Code

```
import numpy as np
import cv2 as cv
import keyboard as kb

cap = cv.VideoCapture(0)
if not cap.isOpened():
    print("Cannot Open Camera")
    exit()
while True:
    ret, frame = cap.read()
    if not ret:
        print("Can't receive frame (stream end?), Exiting ...")
        break

    cv.imshow('frame', frame)
    if cv.waitKey(1) == ord('q'):
        break
    if(kb.is_pressed('ctrl')):
        break

cap.release()
cv.destroyAllWindows()
```

Task: Take any short Video and Resize it. Try drawing a circle over the region of interest or any other shape.

```
import numpy as np
import cv2 as cv
import keyboard as kb

cap = cv.VideoCapture(r"C:\Users\jasti\Downloads\videoplayback.mp4")

if not cap.isOpened():
    print("Cannot Open Camera")
    exit()

while True:
```

```
ret, frame = cap.read()
if not ret:
    print("Can't receive frame (stream end?), Exiting ...")
    break

cv.ellipse(frame, (240, 180), (100, 100), 0, 0, 360, (0, 0, 0), 8)

cv.imshow('frame', frame)

if cv.waitKey(1) == ord('q'):
    break

if (kb.is_pressed('ctrl')):
    break

cap.release()
cv.destroyAllWindows()
```

Day 2 - Session 2

Image Segmentation

- Splitting the image into different parts/segments based on characteristics of pixels.

Clustering

- Unsupervised learning to group data points based on similarity.
- Clustering in images are based on pixel similarity. Plot of each of these pixels in an RGB space.

K-Means Clustering

- Pick K random points (centroids).
- Assign each point to a centroid.
- Take the mean of coordinates of each cluster to be new centroid, discard the old centroid.

- Therefore centroids keep changing. If between two iterations centroid value does not change or changes very little, then it is said that the centroid is STABILISED.

- Distortion cost function-

$$J = \sum_{i=1}^m (x_i - \mu_{ci})^2$$

- Elbow Method to choose K -

1. Start with K = 1.
2. Find Distortion Cost function.
3. Plot on graph.
4. Continue for k = 2, 3, 4...

- The value of K for which J is minimum is the optimum value of K.

- Drawbacks of K means-

1. Poor Initialization

Solution for this is to choose k (fixed) random centroids multiple times (around 10) and choose the set of centroids among the 10 trials such that J is minimum.

2. High Computation Cost

Scatter Plot (MATPLOTLIB) -

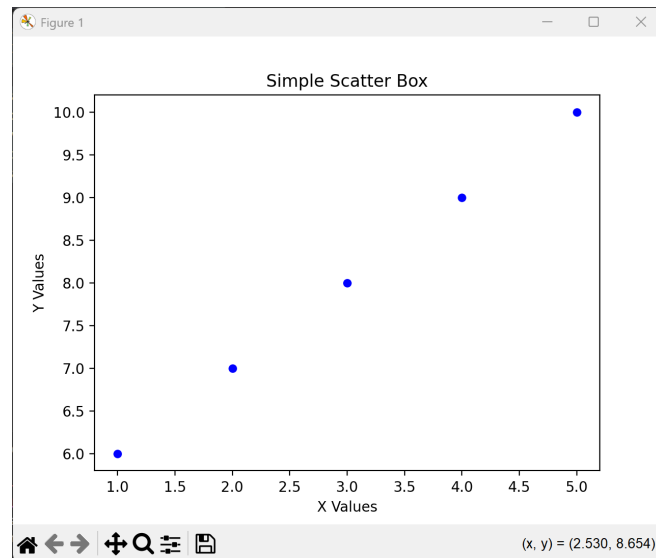
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

plt.scatter(x, y, s=100, marker='.', c='blue')

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Simple Scatter Box")

plt.show()
```



Scatter Plot for Random points in a 2D Array -

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

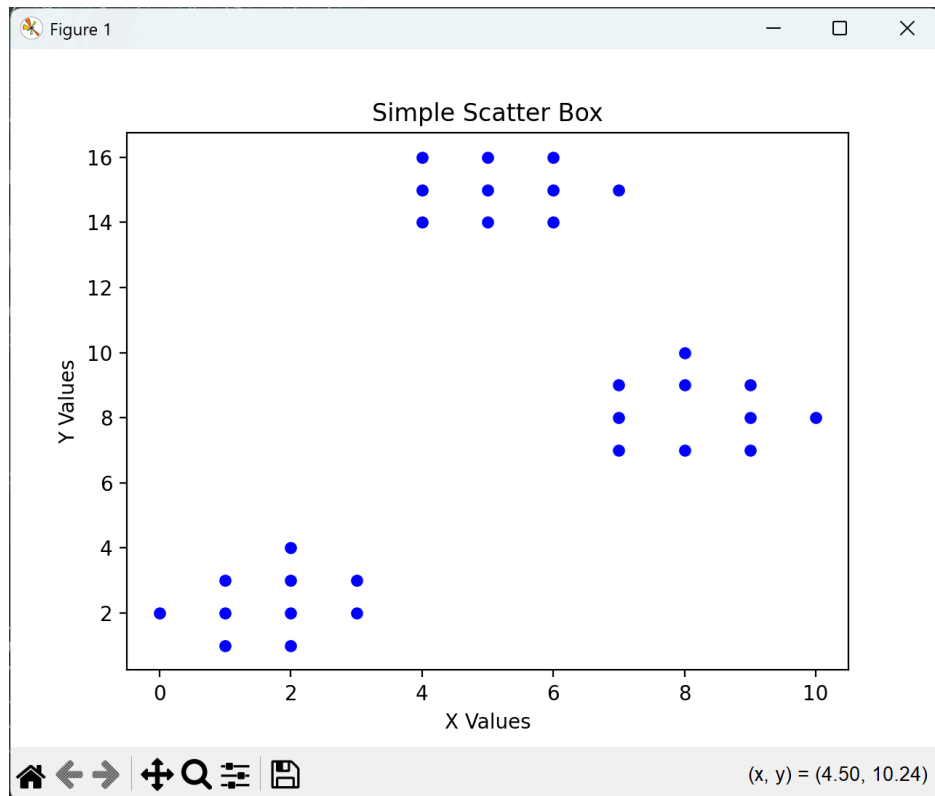
points = np.array([
    [9, 7], [2, 3], [6, 15], [8, 7], [1, 3],
    [5, 14], [7, 9], [3, 2], [4, 15], [2, 2],
    [8, 9], [1, 1], [6, 14], [5, 15], [7, 7],
    [9, 8], [4, 14], [2, 4], [0, 2], [8, 10],
    [5, 16], [3, 3], [10, 8], [7, 8], [1, 2],
    [6, 16], [4, 16], [2, 1], [9, 9], [7, 15]
])

x = points[:, 0]
y = points[:, 1]

plt.scatter(x, y, s=100, marker='.', c='blue')

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Simple Scatter Box")

plt.show()
```



Task: Write a program to execute K-means clustering of the above set of points.

```
import numpy as np
import matplotlib.pyplot as plt

points = np.array([
    [9, 7], [2, 3], [6, 15], [8, 7], [1, 3],
    [5, 14], [7, 9], [3, 2], [4, 15], [2, 2],
    [8, 9], [1, 1], [6, 14], [5, 15], [7, 7],
    [9, 8], [4, 14], [2, 4], [0, 2], [8, 10],
    [5, 16], [3, 3], [10, 8], [7, 8], [1, 2],
    [6, 16], [4, 16], [2, 1], [9, 9], [7, 15]
])

k = 3

centroids = points[np.random.choice(points.shape[0], k, replace=False)]

for epoch in range(10):
    distances = np.linalg.norm(points[:, np.newaxis] - centroids, axis=2)
```

```

labels = np.argmin(distances, axis=1)

new_centroids = np.array([
    points[labels == i].mean(axis=0) for i in range(k)
])

if np.allclose(centroids, new_centroids):
    break

centroids = new_centroids

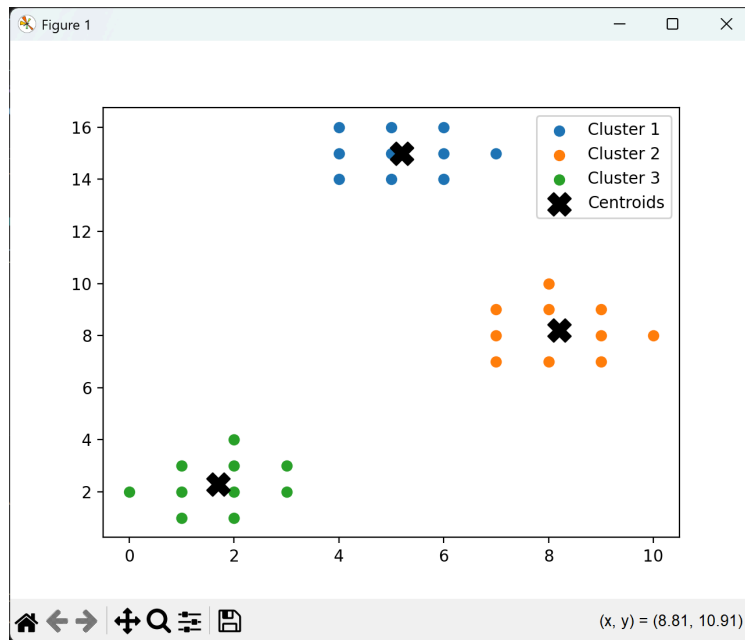
print("Final centroids:\n", centroids)
print("Cluster assignments:", labels)

for i in range(k):
    cluster_points = points[labels == i]
    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        label=f'Cluster {i+1}'
    )

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=200,
    c='black',
    label='Centroids'
)

plt.legend()
plt.show()

```



Task: Write a program to execute K-means clustering for given two images, used RGB coordinates.

```
import numpy as np
import cv2

image = cv2.resize(
    cv2.imread(r"C:\Users\jasti\Downloads\baboon_kmeans.jpeg"),
    (500, 500)
)

clustered = np.zeros_like(image)

k = 7

centroids = [
    image[np.random.randint(0, 500), np.random.randint(0, 500)]
    for _ in range(k)
]

print("Initial centroids:", centroids)

for iteration in range(3):
    clusters = [[] for _ in range(k)]
```

```

print("Iteration", iteration)

for i in range(500):
    for j in range(500):
        dists = [
            np.linalg.norm(c - image[i, j]) for c in centroids
        ]

        kmin = np.argmin(dists)

        clusters[kmin].append((i, j))

old_centroids = centroids.copy()

for i in range(k):
    if len(clusters[i]) > 0:
        rgb_cluster = np.array(
            [image[x, y] for x, y in clusters[i]]
        )

        centroids[i] = np.mean(rgb_cluster, axis=0).astype(int)

        print("Centroid", i, ":", centroids[i])

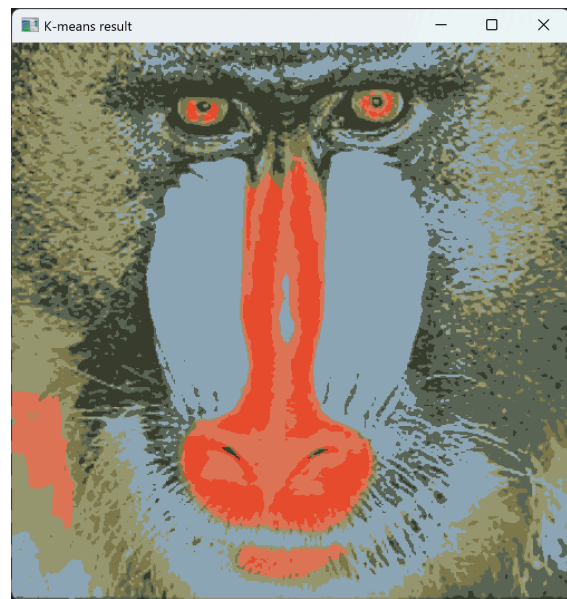
difference = np.abs(
    np.array(old_centroids) - np.array(centroids)
)

if (np.sum(difference) < 10):
    break

for i in range(len(clusters)):
    for x, y in clusters[i]:
        clustered[x, y] = centroids[i]

cv2.imshow("K-means result", clustered)
cv2.imshow("Actual Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



```
import numpy as np
import cv2

image = cv2.resize(
    cv2.imread(r"C:\Users\jasti\Downloads\beach_kmeans.png"),
    (500, 500)
)

clustered = np.zeros_like(image)

k = 10

centroids = [
    image[np.random.randint(0, 500), np.random.randint(0, 500)]
    for _ in range(k)
]

print("Initial centroids:", centroids)

for iteration in range(3):
    clusters = [[] for _ in range(k)]

    print("Iteration", iteration)
```

```

for i in range(500):
    for j in range(500):
        dists = [
            np.linalg.norm(c - image[i, j]) for c in centroids
        ]

        kmin = np.argmin(dists)

        clusters[kmin].append((i, j))

old_centroids = centroids.copy()

for i in range(k):
    if len(clusters[i]) > 0:
        rgb_cluster = np.array(
            [image[x, y] for x, y in clusters[i]]
        )

        centroids[i] = np.mean(rgb_cluster, axis=0).astype(int)

        print("Centroid", i, ":", centroids[i])

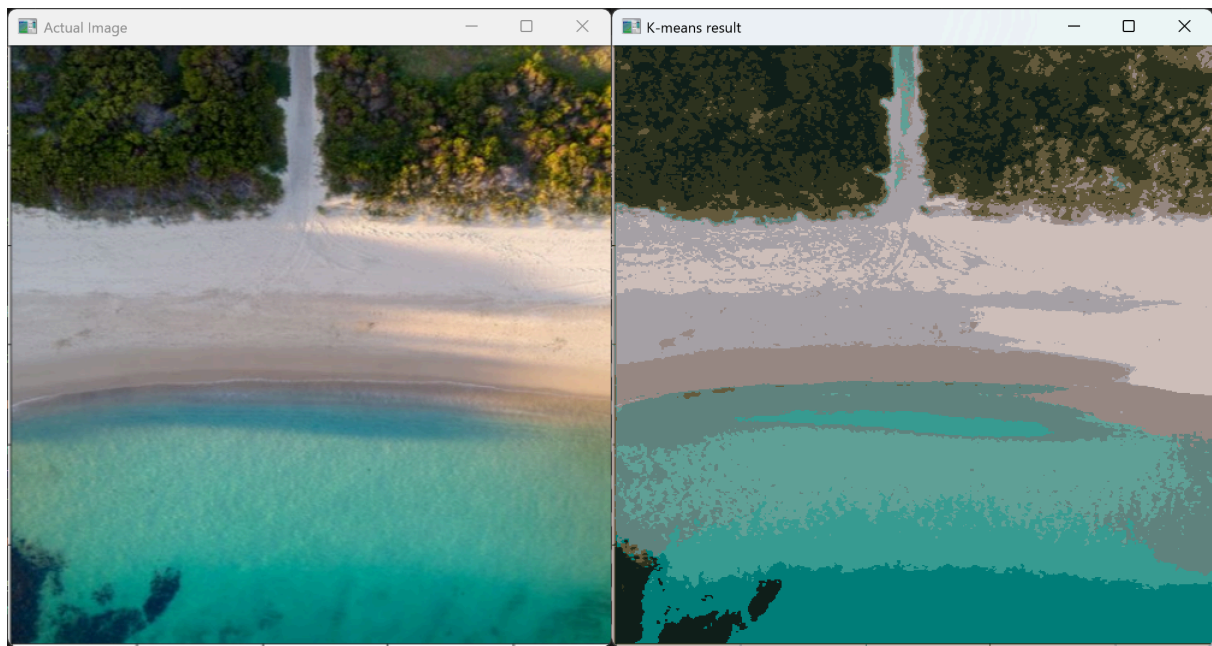
difference = np.abs(
    np.array(old_centroids) - np.array(centroids)
)

if (np.sum(difference) < 10):
    break

for i in range(len(clusters)):
    for x, y in clusters[i]:
        clustered[x, y] = centroids[i]

cv2.imshow("K-means result", clustered)
cv2.imshow("Actual Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



Day 3 - Session 2

Kernels

- Small matrices that are used to find out information about pixels.
- Mixes neighboring pixels to form a new value.

Gaussian Blur

- Blurring isn't always losing information.
- Blurring is important for cancelling noises in the image.
- The value of the final pixel is much closer to the weighted average of the neighboring pixels.
- Steps for Gaussian Blur -
 1. Create a blank canvas of the same size as image one.
 - 2.
- Implementation - Parameters
 1. Source Image
 2. Kernel size

3. Standard deviation of the bell curve to be followed

- Formula to be followed -

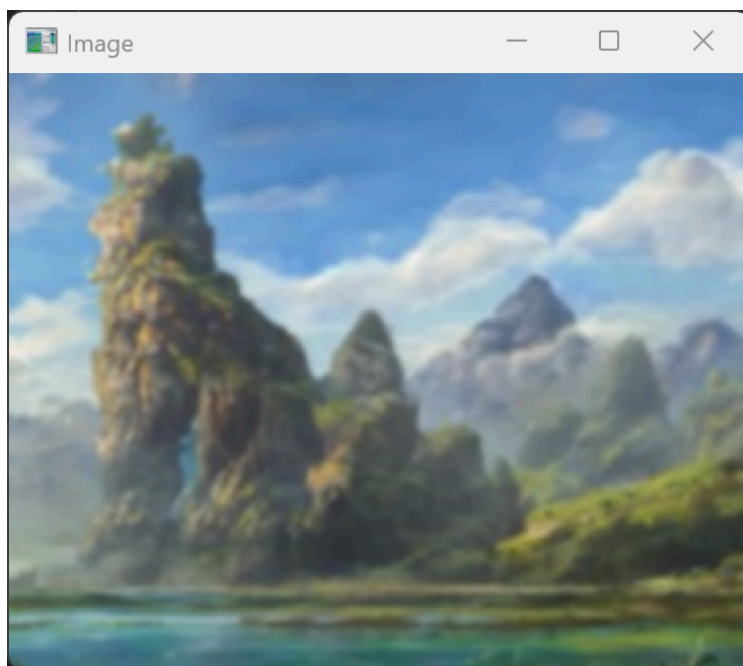
$$K_{size} = 6\sigma + 1$$

```
import cv2 as cv
import numpy as np

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

blur = cv.GaussianBlur(img, (9, 9), 1.5)

cv.imshow("Image", blur)
cv.waitKey(0)
cv.destroyAllWindows()
```



Bilateral Filtering -

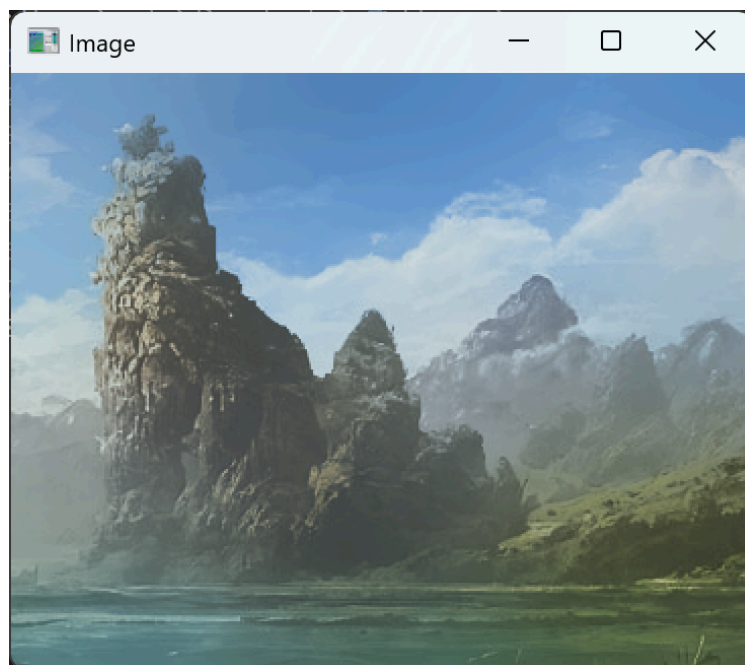
- Better than Gaussian Blur
- Helps in preserving edges.
- Edges are defined as the set of pixels forming a transition layer between large sets of pixels of same intensities. Basically areas of intense transition.

```
import cv2 as cv
import numpy as np

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

blur = cv.bilateralFilter(img, d=100, sigmaColor=75, sigmaSpace=50)

cv.imshow("Image", blur)
cv.waitKey(0)
cv.destroyAllWindows()
```



Edge Detection-

- Process of finding edges in an image. Edges are defined above.

Methods of Edge Detection -

Sobel Method -

- Contains two two kernels - SobelX (Gx) and SobelY(Gy).
- SobelX performs the task of gradient in X direction.
- SobelY performs the task of gradient in Y direction.

- $G = \sqrt{G_x^2 + G_y^2}$

Task: Using Sobel Method, detect the edges of any image.

```
import cv2 as cv
import numpy as np

img = cv.imread(r"C:\Users\jasti\Downloads\scenery.jpeg", cv.IMREAD_GRAY
SCALE)

blur = cv.GaussianBlur(img, (9, 9), 1.5)

sx = cv.Sobel(blur, ddepth=cv.CV_64F, dx=1, dy=0, ksize=3)

sy = cv.Sobel(blur, ddepth=cv.CV_64F, dx=0, dy=1, ksize=3)

mag = np.hypot(sx, sy)

mag = mag / mag.max() * 255

mag = mag.astype(np.uint8)

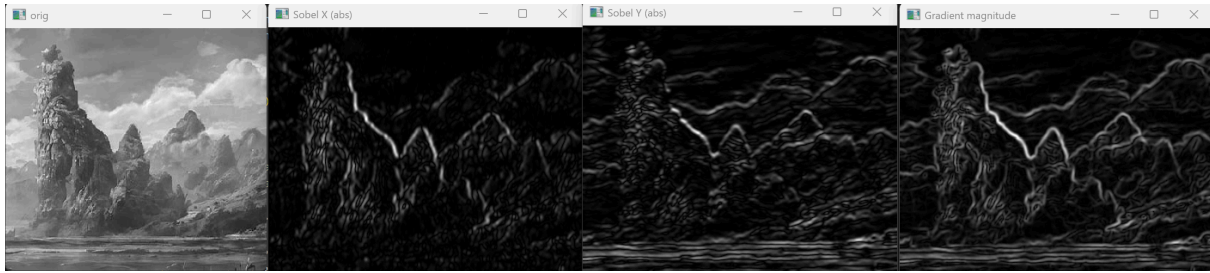
abs_sx = np.absolute(sx)
abs_sy = np.absolute(sy)

abs_sx = (abs_sx / abs_sx.max() * 255).astype(np.uint8)
abs_sy = (abs_sy / abs_sy.max() * 255).astype(np.uint8)

cv.namedWindow("orig", cv.WINDOW_NORMAL)

cv.imshow("orig", img)
cv.imshow("Sobel X (abs)", abs_sx)
cv.imshow("Sobel Y (abs)", abs_sy)
cv.imshow("Gradient magnitude", mag)

cv.waitKey(0)
cv.destroyAllWindows()
```



Laplace Method -

- Kernel form of Laplace Method -

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- This Kernel is equivalent to $\nabla^2 f = f_{xx} + f_{yy}$
- Comes from Taylor Series.
- Corresponds to second derivative.
- In Sobel's method, we see rate of change of intensity for edge. This does not work every time. For example, a canvas with same color whose intensity is changing constantly.

Task: Using Laplace Method, detect the edges of any image.

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

lap = cv2.Laplacian(gray, cv2.CV_64F)

lap = np.uint8(np.absolute(lap))

cv2.imshow("Laplacian Edge Detection", lap)

cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



Gradients -

- $\nabla I = \frac{\delta I}{\delta x} \hat{x} + \frac{\delta I}{\delta y} \hat{y}$
- Assume pixels are as shown below

$$\begin{array}{cc} (x, y) & (1, 0) \\ (0, 1) & (1, 1) \end{array}$$

- $\text{grad } I_x(x, y) = f(x + 1, y) - f(x, y)$
- $\text{grad } I_y(x, y) = f(x, y + 1) - f(x, y)$
- In order to avoid this, since these aren't really accurate, we use Sobel method to find gradient.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- $G = \sqrt{G_x^2 + G_y^2}$

Canny Edge Detection -

- Steps involved -
 1. Perform noise reduction using Gaussian blur.
 2. Apply Sobel's method to find out gradient.
 3. Perform Non-max oppression
 - Keep two threshold values T_0 and T_1 .
 - Detect Strong edge points and weak edge points
- Some NumPy functions that are helpful -
 1. $np.hypot(I_x, I_y)$
 2. $G = G/G.max() * 255$
 3. $theta = np.arctan2(I_y, I_x)$

Task: Using Canny Edge Detection, detect edges of any image

```
import cv2

img = cv2.imread(r"C:\Users\jasti\Downloads\scenery.jpeg", cv2.IMREAD_GRAYSCALE)

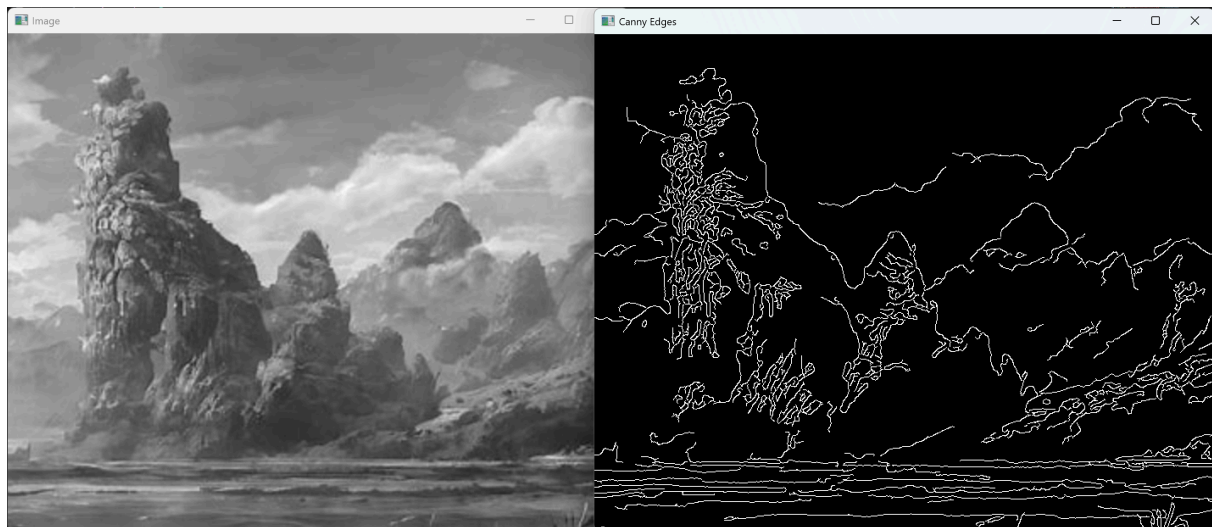
img = cv2.resize(img, (732, 586))

blurred = cv2.GaussianBlur(img, (5, 5), 1.4)

edges = cv2.Canny(blurred, 20, 100)

cv2.imshow('Image', img)
cv2.imshow('Canny Edges', edges)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Hough Transform -

- Sometimes edges are broken. To prevent those broken edges we use Hough Transform.
- Basic Principle -
 - $y = mx + c$
 - $c = -mx + y$
 - $\implies$ Every point in the XY plane corresponds to a line in the CM plane.
 - $\implies$ If there are two points (x_1, y_1) and (x_2, y_2) , the intersection point of these two lines in CM plane will correspond to the line in the XY plane containing the two points.
 - Initialize an ACCUMULATOR array.

Day 4 - Session 1

Machine Learning for Computer Vision

- We write general algorithms and feed data into it such that the computer learns from the data.
- Prediction function is a linear combination of parameters

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

θ s are called parameters

x_i s are called features

- Loss function -

We need to make the output of $h(x)$ as close to the actual value of the target variable.

$$J(\theta) = \frac{1}{2m} \sum_i (h_{\theta}(x)^i - y^i)^2$$

Linear regression is designed to reduce the value of the loss function.

Optimizing parameters -

- Find the optimal values of theta for which loss is minimized.
- 2 Methods are used -
 1. Least Squared Method
 2. Gradient Descent Method

Gradient Descent Method -

- Δx depends on $\frac{dy}{dx}$
- $x_{t+1} - x_t = -\alpha \frac{dy}{dx}$
- In terms of Machine Learning, y is Loss function
- x is theta.

$$\implies \theta_{t+1} = \theta_t - \alpha \frac{dL}{d\theta}$$

- $\frac{dL}{d\theta_1} = \frac{1}{2m} \sum_i 2x_1(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$
- α is called the Learning Rate.

Task: Write the code for linear regression using gradient descent method on the California Housing Dataset.

```
from sklearn.model_selection import train_test_split
import numpy as np
from numpy.linalg import inv

data = np.genfromtxt("housing.csv", delimiter=",", skip_header=1)
X = data[:, :-1]
y = data[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size = 0.2, random_state = 42
)

class LinearRegressionGDloops:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.theta = None
        self.mse_history_train = []
        self.mse_history_val = []

    def fit(self, X_train, y_train, X_test, y_test):
        # Add intercept term to X
        X_train_b = self.add_intercept(X_train)
        X_test_b = self.add_intercept(X_test)
        m, n = X_train_b.shape
        self.theta = np.zeros(n)

        # Gradient Descent
        for _ in range(self.iterations):
            predictions_train = self.predict(X_train_b)
            gradients = (2/m)* X_train_b.T.dot(predictions_train - y_train)
            self.theta -= self.learning_rate*gradients

            # Compute and store MSE for training and validation sets
            mse_train = self.compute_mse(y_train, predictions_train)
            self.mse_history_train.append(mse_train)

            predictions_test = self.predict(X_test_b)
            mse_test = self.compute_mse(y_test, predictions_test)
            self.mse_history_val.append(mse_test)

    def predict(self, X):
        y_pred = []
        for i in range(X.shape[0]):
            y_pred.append(np.dot(X[i, :], self.theta))
        return np.array(y_pred)

    def compute_mse(self, y_true, y_pred):

```

```

    return np.mean((y_true - y_pred) ** 2)

def add_intercept(self, X):
    return np.c_[np.ones(X.shape[0]), X] # Prepends a column of ones to the
array X, often used to add an intercept term to the feature matrix.

# Helper method to get the history for plotting
def get_mse_history(self):
    return [self.mse_history_train, self.mse_history_val]

# Compute parameters using the normal equation
def normal_equation(X, y):
    X_b = np.c_[np.ones(X.shape[0]), X] # Add intercept term
    return inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)

# Create and fit the model
model = LinearRegressionGDloops(learning_rate=0.01, iterations=1000)
model.fit(X_train, y_train, X_test, y_test)

```

Task: Write a program to implement Gradient Descent on a Simple Dataset and find the best fit line

```

import numpy as np
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 3.7, 3.2, 4.8, 6.1])

m = 0.0
b = 0.0
lr = 0.01
epochs = 100

for i in range(epochs):
    y_pred = m * X + b
    error = y_pred - y

    dm = (2/len(X)) * np.dot(error, X)
    db = (2/len(X)) * np.sum(error)

```

```

m -= lr * dm
b -= lr * db

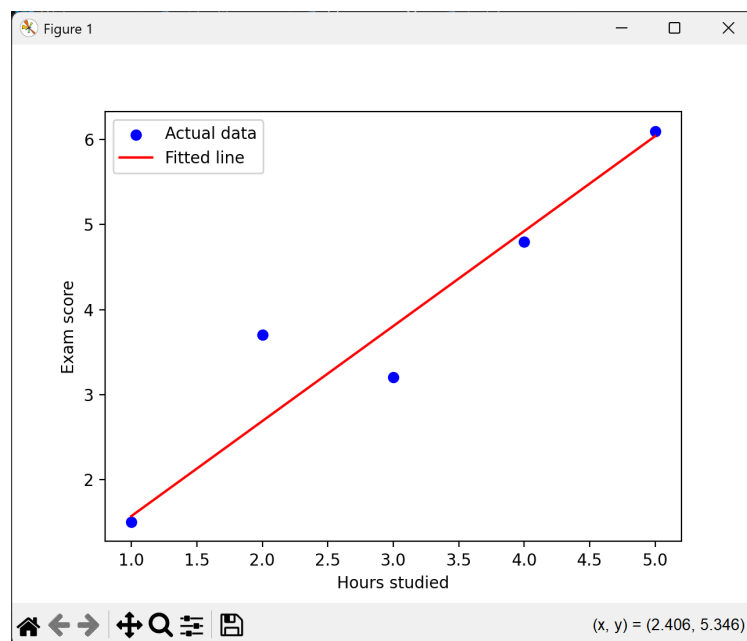
print(f"\nFinal model: y = {m:.2f}x + {b:.2f}")

plt.scatter(X, y, color='blue', label="Actual data")

plt.plot(X, m * X + b, color='red', label="Fitted line")

plt.xlabel("Hours studied")
plt.ylabel("Exam score")
plt.legend()
plt.show()

```



Day 4 - Session 2

- Representation of Vectors as Matrices was introduced.
- Topic of lecture: Logistic Regression

Logistic Regression

- Fits the output between zero and one.
- Special function is used, known as sigmoid function.

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- Hence we define the following

$$\hat{y} = \frac{1}{1+e^{-(wx+b)}}$$

$$\hat{y} = \frac{1}{1 + e^{-(wx+b)}}$$

$$\frac{\partial \hat{y}}{\partial w} = \frac{x e^{-(wx+b)}}{(1 + e^{-(wx+b)})^2}$$

Task: Write a program to implement Gradient Descent on a Simple Dataset for Logistic Regression.

```
import numpy as np
import math

X = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 3.7, 3.2, 4.8, 6.1])

y_bin = np.array([1 if v > 3 else 0 for v in y], dtype=float)

w = 0.0
b = 0.0
lr = 0.1

for _ in range(1000):
    dw = 0.0
    db = 0.0

    for i in range(len(X)):
        z = w * X[i] + b
        pred = 1 / (1 + math.exp(-z))
        dw += (pred - y_bin[i]) * X[i]
        db += (pred - y_bin[i])

    w -= lr * dw / len(X)
    b -= lr * db / len(X)

print(round(w, 4), round(b, 4))
```

```

predictions = []
for i in range(len(X)):
    z = w * X[i] + b
    predictions.append(round(1 / (1 + math.exp(-z)), 4))

print(predictions)

```

```

2.717 -3.7029
[0.2717, 0.8496, 0.9884, 0.9992, 0.9999]

```

Universal Approximation Theorem

- **Claim:** Circle $\approx$ Triangle
- Call a Triangle an approximation of a circle.

Neural Networks

- Uses composition of Logistic sigmoid function several times to approximate the curve to the best fit model to perform an operation on a dataset.
- Activation functions -
 1. $\sigma(x)$
 2. $\tanh(x)$
 3. $ReLU(x)$

$$ReLU(x) = \max(0, x) = \begin{cases} x, & 0 \leq x \\ 0, & x \leq 0 \end{cases}$$

- Problem with $ReLU(x)$ is that once the value becomes negative it's impossible to go in the other direction.
- Chain Rule is very important in Neural networks.
- Sometimes we store some derivative values in variables to reduce computational power.
- Read about vanishing/exploding gradient problem

- To fix the ReLU Problem, engineers came up with a function called Leaky ReLU function.
 - $\text{Leaky ReLU}(x) = \max(\epsilon x, x)$, where ϵ is a very small value.
- Neural Networks are called Fully Connected Layers (FCs in short)

Convolutional Neural Networks

Parameters and Hyperparameters

- Parameters include weights and biases.
- Hyper Parameters include Output size, Number of Hidden Layers, Size of hidden layers

Introduction of a new Loss function -

$$\text{Cross Entropy Loss} = \sum_i [y_i \log \hat{y}_i]$$

Kernels Continued

- Always choose the dimensions of Kernels to be odd dimensional.
- When a kernel slides onto an image of dimension I, dimension of output image will be

$$Dim = \frac{I - K + 2P}{S} + 1$$

Where P is the Padding

S is the Stride

K is dimension of Kernel

I is dimension of Image

- Softmax →

$$\phi(y_i) = \frac{e^{\hat{y}_i}}{\sum_i^n e^{\hat{y}_i}}$$

- Overfitting: Process of models memorizing the data-set instead of getting trained.
 - Training accuracy is almost 100%.

- Testing accuracy is around 25%.

Training Models

Gradient Descent

- Batch Gradient
- Stochastic
- Batch Stochastic
- Momentum
- Nesterov Accelerated
- Adam
- AdaGrad

Day 5 - Session 1

How to Represent Graphs

1. Edge List
2. Adjacency List
3. Adjacency Matrix

Edge List -

- Consists of Nodes and Edges.
- Stores tuples of the form (Start node, End node, Weight)

Adjacency List -

- Array of size V , in which each Index holds a linked list containing vertices that the vertex at that index is connected to.
- Most popular representation.
- Eg. If node 0 has 1 and 2 as neighbors, then index 0 would contain [1, 2]

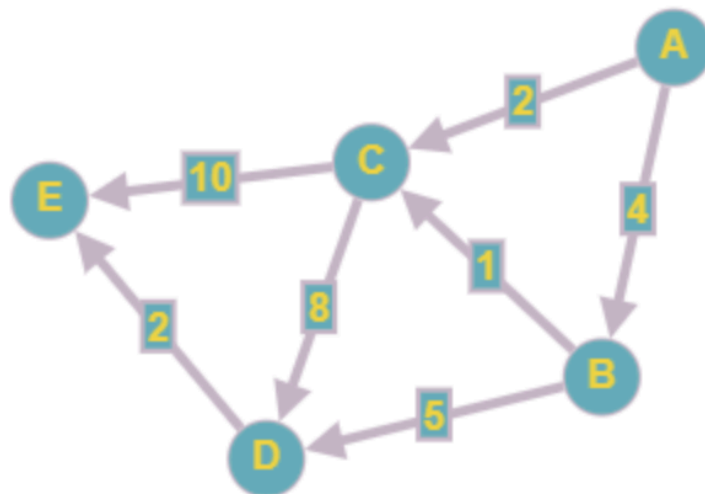
Adjacency Matrix -

- Contains only 0 and 1 for unweighted graph.
- Contains positive integers for weighted graph.
- Rows contain start nodes, Columns are destination nodes.
- Non-directional Graph $\implies$ Symmetric Matrix
- Directional Graph $\implies$ Unsymmetric Matrix

Graph Traversal -

- Systematic process of visiting all vertices of a graph in a particular order.

Graph to be used for BFS, DFS and Dijkstra's Algorithm Implementation



Edge List Representation of Above Graph

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2], ['C', 'E', 10]]
```

Breadth First Search -

- Check Level by level.
- Explore widely before going deep.

Algorithm -

1. Mark all vertices as unvisited.

2. Create an empty queue Q and enqueue s. Mark s as under-visit.
3. While Q $\neq$ empty:
 - (i) Dequeue a vertex u from Q and mark u visited.
 - (ii) For each unvisited neighbor v of u:
 - (a) Mark v as under-visit, set u as its parent, and enqueue v.

Task: Write a general code to implement BFS algorithm

```
graph = {
    'A': [],
    'B': [],
    'C': [],
    'D': [],
    'E': [],
    'F': []
}

start = 'A'

visited = set()

queue = [start]

while queue:
    node = queue.pop(0)

    if node not in visited:
        print(node)
        visited.add(node)

        for n in graph[node]:
            if n not in visited:
                queue.append(n)
```

Task: Implement BFS Algorithm on the given graph.

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]
```

```

adj = {}

for e in graph:
    u = e[0]
    v = e[1]
    if u not in adj:
        adj[u] = []
    adj[u].append(v)

start = 'A'

visited = []
queue = []

visited.append(start)
queue.append(start)

while queue:
    node = queue.pop(0)
    print("Popped:", node)

    if node in adj:
        for n in adj[node]:
            if n not in visited:
                visited.append(n)
                queue.append(n)

print(visited)
print(adj)

```

```

Popped: A
Popped: B
Popped: C
Popped: D
Popped: E
['A', 'B', 'C', 'D', 'E']
{'A': ['B', 'C'], 'B': ['C', 'D'], 'C': ['D', 'E'], 'D': ['E']}

```

Depth First Search -

- Keep moving until you hit a wall.
- Backtrack when you hit a dead-end or visited node.

Algorithm -

1. Mark all vertices as unvisited
2. Create an empty stack S and push s ; marks s as under-visit
3. while $S \neq \text{empty}$ do
 - Let u be the top of S
 - if u has an unvisited neighbor v then: //Visit v along the edge (u,v)
 - Set $\pi[v]$ as u , mark v as under-visit, and push v onto S
 - else //Backtrack to parent u along the edge (v,u)
 - Pop u from S and mark it visited

Task: Write a general code to implement DFS algorithm

```
graph = {
    'A': [],
    'B': [],
    'C': [],
    'D': [],
    'E': [],
    'F': []
}

start = 'A'

visited = set()
stack = [start]

while stack:
    node = stack.pop()
    if node not in visited:
        print(node)
        visited.add(node)
        for n in reversed(graph[node]):
```

```
if n not in visited:
    stack.append(n)
```

Task: Implement DFS Algorithm on the given graph.

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]
```

```
adj = {}
```

```
for e in graph:
```

```
    u = e[0]
```

```
    v = e[1]
```

```
    if u not in adj:
```

```
        adj[u] = []
```

```
    adj[u].append(v)
```

```
visited = []
```

```
def dfs(node):
```

```
    visited.append(node)
```

```
    print(node)
```

```
    if node in adj:
```

```
        for n in adj[node]:
```

```
            if n not in visited:
```

```
                dfs(n)
```

```
dfs('A')
```

```
print(adj)
```

```
A
B
C
D
E
{'A': ['B', 'C'], 'B': ['C', 'D'], 'C': ['D', 'E'], 'D': ['E']}
```

Dijkstra's Algorithm

- Greedy Algorithm.
- Assign every node a tentative distance.
 - Start Node $\rightarrow 0$
 - All other Nodes $\rightarrow \infty$
- Maintain set of unvisited nodes.
- Choose unvisited node with shortest distance.
- Mark it visited $\rightarrow$ its shortest distance is now final.
- Relax all outgoing edges from that node: $d[v] = \min(d[v], d[u] + w(u, v))$

Algorithm -

Initialization -

1. For all vertices $v \in V$:
 - $\text{dist}[v] = \infty$
 - $\text{parent}[v] = \text{null}$
2. Set the source vertex:
 - $\text{dist}[s] = 0$
3. Insert the source into the priority queue:
 - push $(0, s)$ into priority queue

Main Loop -

While the priority queue is **not empty**:

1. Extract the node with minimum distance:
 $(du, u) = \text{extract_min}()$
2. If u is already visited:
 - Continue to the next iteration
3. Mark u as visited
4. For each neighbor v of u :
 - If a shorter path is found:

```
if dist[u] + w(u, v) < dist[v]:
    dist[v] = dist[u] + w(u, v)
    parent[v] = u
    push (dist[v], v)
```

Outputs -

- `dist[v]` → Shortest distance from source `s` to vertex `v`
- `parent[v]` → Used to reconstruct the shortest path

Task: Implement Dijkstra's Algorithm on the given graph.

```
graph = [['A', 'B', 4], ['A', 'C', 2], ['B', 'C', 1], ['B', 'D', 5], ['C', 'D', 8], ['D', 'E', 2],
['C', 'E', 10]]
```

```
adj = {}
```

```
for u, v, w in graph:
    if u not in adj:
        adj[u] = []
    adj[u].append((v, w))
```

```
dist = {}
prev = {}
nodes = []
```

```
for e in graph:
    dist[e[0]] = 9999
    dist[e[1]] = 9999
    prev[e[0]] = None
    prev[e[1]] = None
    if e[0] not in nodes:
        nodes.append(e[0])
    if e[1] not in nodes:
        nodes.append(e[1])
```

```
dist['A'] = 0
```

```
while nodes:
```

```

u = nodes[0]
for n in nodes:
    if dist[n] < dist[u]:
        u = n
nodes.remove(u)

if u in adj:
    for v, w in adj[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            prev[v] = u

path = []
cur = 'E'

while cur:
    path.insert(0, cur)
    cur = prev[cur]

print("Path:", path)
print("Distance:", dist['E'])

```

```

Path: ['A', 'B', 'D', 'E']
Distance: 11

```

Day 5 - Session 2

SLAM

- Simultaneous Localization and Mapping
- Assume a Robot that only knows its start position and final destination but does not know the route to it.
- It considers errors that the robot comes across when using its LiDAR.

Task: Write the general code for A* Algorithm (When diagonal movement is not allowed)

```

grid = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,0,0],
    [0,0,0,0,0]
]

start = (0, 0)
goal = (4, 4)

open_list = [start]
came_from = {}
g = {start: 0}
f = {start: abs(start[0]-goal[0]) + abs(start[1]-goal[1])}

def neighbors(n):
    x, y = n
    res = []
    for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 5 and 0 <= ny < 5 and grid[ny][nx] == 0:
            res.append((nx, ny))
    return res

while open_list:
    cur = open_list[0]
    for n in open_list:
        if f[n] < f[cur]:
            cur = n

    if cur == goal:
        break

    open_list.remove(cur)

    for n in neighbors(cur):
        temp = g[cur] + 1
        if n not in g or temp < g[n]:

```

```

    came_from[n] = cur
    g[n] = temp
    f[n] = g[n] + abs(n[0]-goal[0]) + abs(n[1]-goal[1])

    if n not in open_list:
        open_list.append(n)

path = []
cur = goal
while cur in came_from:
    path.insert(0, cur)
    cur = came_from[cur]
path.insert(0, start)

dist = len(path) - 1

print("Path:", path)
print("Path Length:", dist)

```

```

Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Path Length: 8

```

Task: Write the general code for A* Algorithm (When diagonal movement is allowed)

```

grid = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,0,0],
    [0,0,0,0,0]
]

start = (0, 0)
goal = (4, 4)

open_list = [start]
came_from = {}
g = {start: 0}

```

```

f = {start: abs(start[0]-goal[0]) + abs(start[1]-goal[1])}

def neighbors(n):
    x, y = n
    res = []
    for dx, dy in [(1,0),(-1,0),(0,1),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 5 and 0 <= ny < 5 and grid[ny][nx] == 0:
            res.append((nx, ny))
    return res

while open_list:
    cur = open_list[0]
    for n in open_list:
        if f[n] < f[cur]:
            cur = n

    if cur == goal:
        break

    open_list.remove(cur)

    for n in neighbors(cur):
        if abs(n[0]-cur[0]) + abs(n[1]-cur[1]) == 2:
            step = 1.4
        else:
            step = 1

        temp = g[cur] + step

        if n not in g or temp < g[n]:
            came_from[n] = cur
            g[n] = temp
            f[n] = g[n] + abs(n[0]-goal[0]) + abs(n[1]-goal[1])

            if n not in open_list:
                open_list.append(n)

path = []

```

```

cur = goal
dist = 0

while cur in came_from:
    p = came_from[cur]

    if abs(p[0]-cur[0]) + abs(p[1]-cur[1]) == 2:
        dist += 1.4
    else:
        dist += 1

    path.insert(0, cur)
    cur = p

path.insert(0, start)

print("Path:", path)
print("Path Length:", dist)

```

```

Path: [(0, 0), (1, 0), (2, 1), (2, 2), (3, 3), (4, 4)]
Path Length: 6.199999999999999

```

Task: Take an image, convert it to black and white and detect the edges using Canny Edge Detection. Pick two points and implement A* Algorithm to find the optimal path between the points. Consider the edges to be obstacles.

```

import cv2
import numpy as np

img = cv2.imread(r"C:\Users\jasti\Downloads\maze_Astar.png")

img = cv2.resize(img, (400, 400))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 50, 150)

```

```

ys, xs = np.where(edges > 0)

min_x, max_x = xs.min(), xs.max()
min_y, max_y = ys.min(), ys.max()

mask = np.ones_like(edges) * 255

mask[min_y:max_y, min_x:max_x] = edges[min_y:max_y, min_x:max_x]

edges = mask

grid = edges / 255

start = (30, 30)
end = (380, 380)

open_list = [(0, start)]
came_from = {}
g_score = {start: 0}

while open_list:
    open_list.sort()
    current = open_list.pop(0)[1]

    if current == end:
        break

    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:

        neighbor = (current[0] + dx, current[1] + dy)

        if 0 <= neighbor[0] < 400 and 0 <= neighbor[1] < 400:

            if grid[neighbor[1], neighbor[0]] == 1:
                continue

            temp_g = g_score[current] + 1

            if neighbor not in g_score or temp_g < g_score[neighbor]:

```

```

g_score[neighbor] = temp_g

h = abs(neighbor[0] - end[0]) + abs(neighbor[1] - end[1])

f_score = temp_g + h

open_list.append((f_score, neighbor))
came_from[neighbor] = current

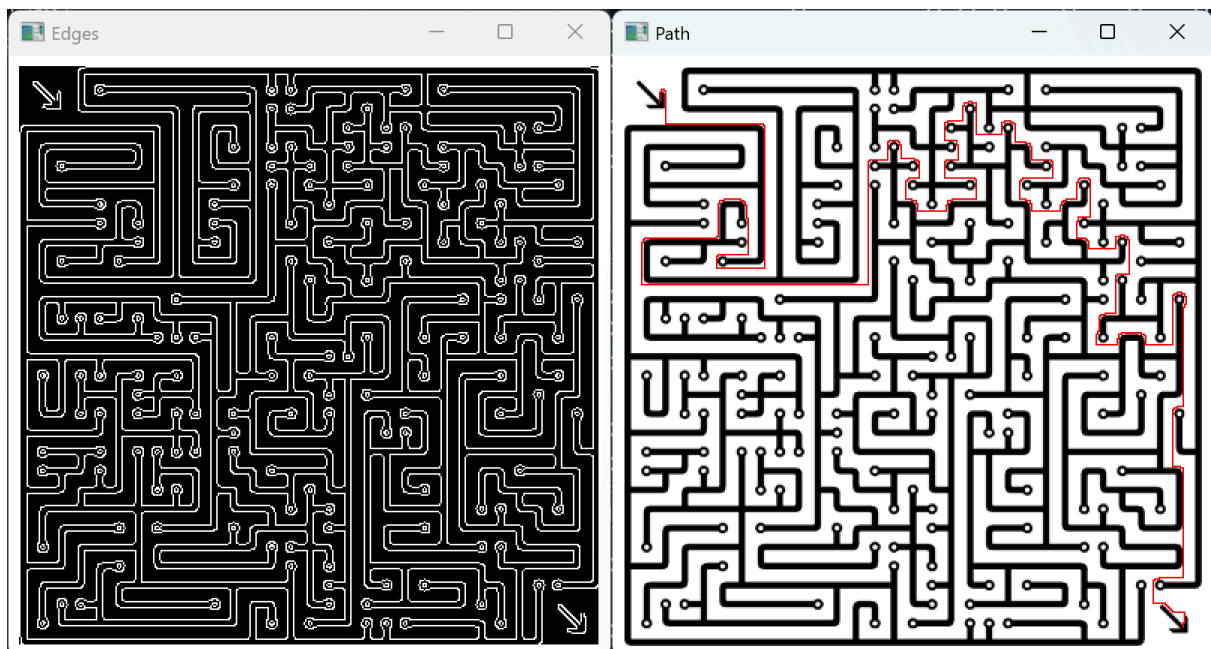
curr = end
while curr in came_from:
    img[curr[1], curr[0]] = [0, 0, 255]
    curr = came_from[curr]

cv2.imshow('Edges', edges)

cv2.imshow('Path', img)

cv2.waitKey(0)
cv2.destroyAllWindows()

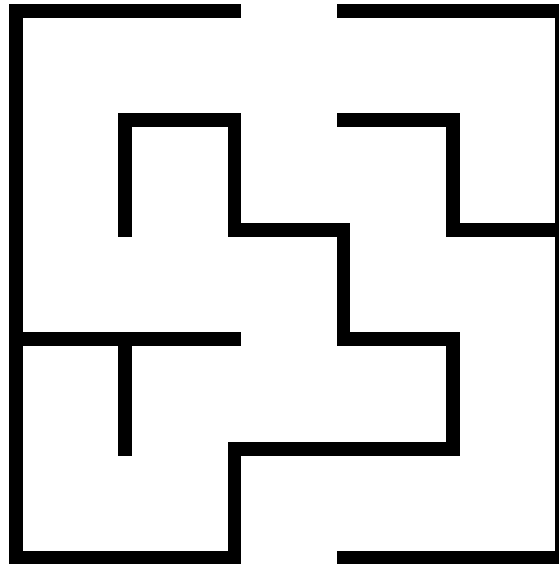
```



Day 6 - Session 1

- Computationally expensive to implement A* on image.
- Therefore we don't consider every pixel as a node. We try to find different optimal nodes on the image we are dealing, instead of all pixels.

Perform the following tasks to implement algorithms on this given maze.



Probabilistic Roadmap (PRM)

- Configurational space, free space and obstacles are introduced.
- Nodes must be on the free space.
- Generate a random node, and check if it lies in the free space. If in free space, accepted. If it intersects with obstacle, reject and generate a new node.
- Add accepted nodes to the graph. Connect every new node to its k closest nodes.
- The value of k must be optimal, neither too high nor too less.
- If any edge is intersecting with obstacles, discard the edge.
- This will create a graph. Then apply A* on it.

Task: Implement PRM on the given maze.

```
import cv2
import numpy as np
import random
```

```

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")

img = cv2.resize(img, (400, 400))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

start = (30, 370)
end = (200, 395)
nodes = [start, end]

while len(nodes) < 100:
    rx = random.randint(5, 395)
    ry = random.randint(5, 395)
    if thresh[ry, rx] == 255:
        nodes.append((rx, ry))

adj = {i: [] for i in range(len(nodes))}

for i in range(len(nodes)):
    for j in range(i + 1, len(nodes)):
        p1 = nodes[i]
        p2 = nodes[j]

        dist = ((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)**0.5

        if dist < 100:

            line_img = np.zeros((400, 400), dtype=np.uint8)

            cv2.line(line_img, p1, p2, 255, 1)

            if not np.any(cv2.bitwise_and(line_img, cv2.bitwise_not(thresh))):
                adj[i].append(j)
                adj[j].append(i)

queue = [(0, [0])]

```

```

visited = set()
path = []

while queue:
    queue.sort()
    dist, current_path = queue.pop(0)
    last_node = current_path[-1]

    if last_node == 1:
        path = current_path
        break

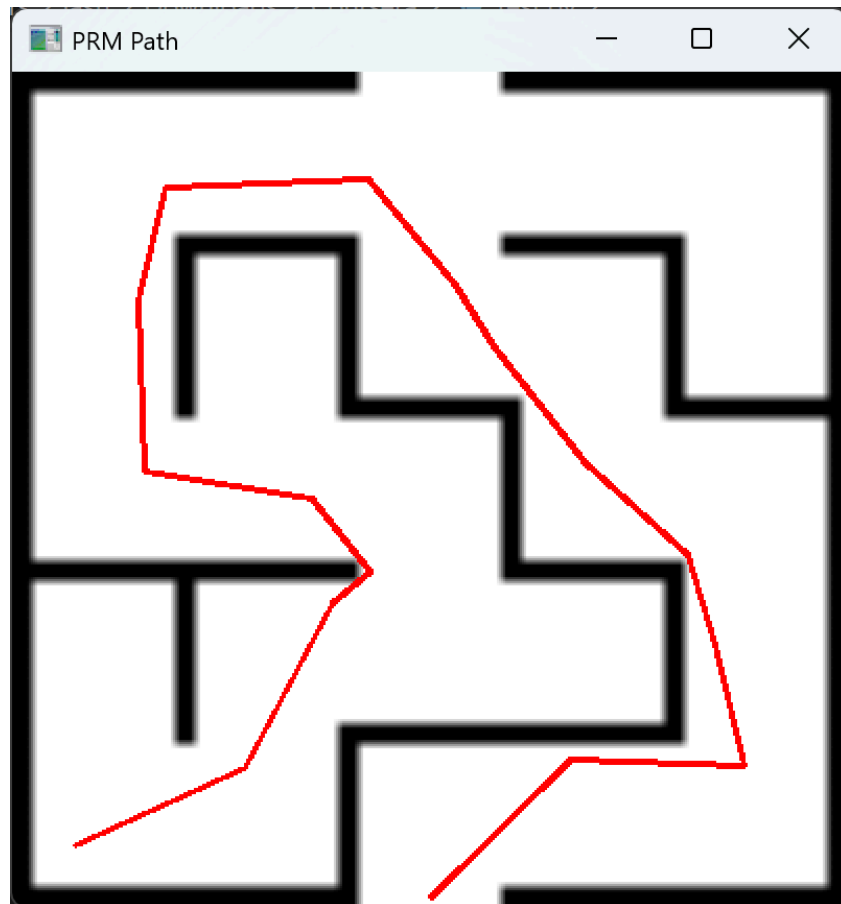
    if last_node not in visited:
        visited.add(last_node)
        for neighbor in adj[last_node]:
            new_dist = dist + (
                (nodes[last_node][0]-nodes[neighbor][0])**2 +
                (nodes[last_node][1]-nodes[neighbor][1])**2
            )**0.5

            queue.append((new_dist, current_path + [neighbor]))

for i in range(len(path)-1):
    cv2.line(img, nodes[path[i]], nodes[path[i+1]], (0, 0, 255), 2)

cv2.imshow('PRM Path', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



Rapidly-exploring Random Trees (RRT)

- Start from the source by adding a node there.
- Generate a point. Check if there are no obstacles between it and its nearest node. If distance is greater than THRESHOLD DISTANCE.
- If distance greater than threshold distance, shift point along the line such that distance equals threshold value.
- Now that's the new node. Check if there is an obstacle between this new node and the nearest node.
- If there is an obstacle, DISCARD THE NODE (not just the edge).
- Input to the computer are number of nodes to be generated and threshold distance.

Task: Implement RRT on the given maze.

```
import cv2
import numpy as np
import random
```

```

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")

img = cv2.resize(img, (400, 400))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, bw = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

start = (30, 370)
goal = (200, 395)

nodes = [start]
parent = {start: None}
step = 20

def free(p):
    return bw[p[1], p[0]] == 255

def dist(a, b):
    return ((a[0]-b[0])**2 + (a[1]-b[1])**2) ** 0.5

def steer(a, b):
    d = dist(a, b)
    if d < step:
        return b
    x = int(a[0] + step * (b[0]-a[0]) / d)
    y = int(a[1] + step * (b[1]-a[1]) / d)
    return (x, y)

def collision(a, b):
    for i in range(21):
        x = int(a[0] + (b[0]-a[0]) * i / 20)
        y = int(a[1] + (b[1]-a[1]) * i / 20)
        if bw[y, x] == 0:
            return True
    return False

found = False

```

```

for _ in range(5000):
    rand = (random.randint(0, 399), random.randint(0, 399))

    if not free(rand):
        continue

    near = nodes[0]
    for n in nodes:
        if dist(n, rand) < dist(near, rand):
            near = n

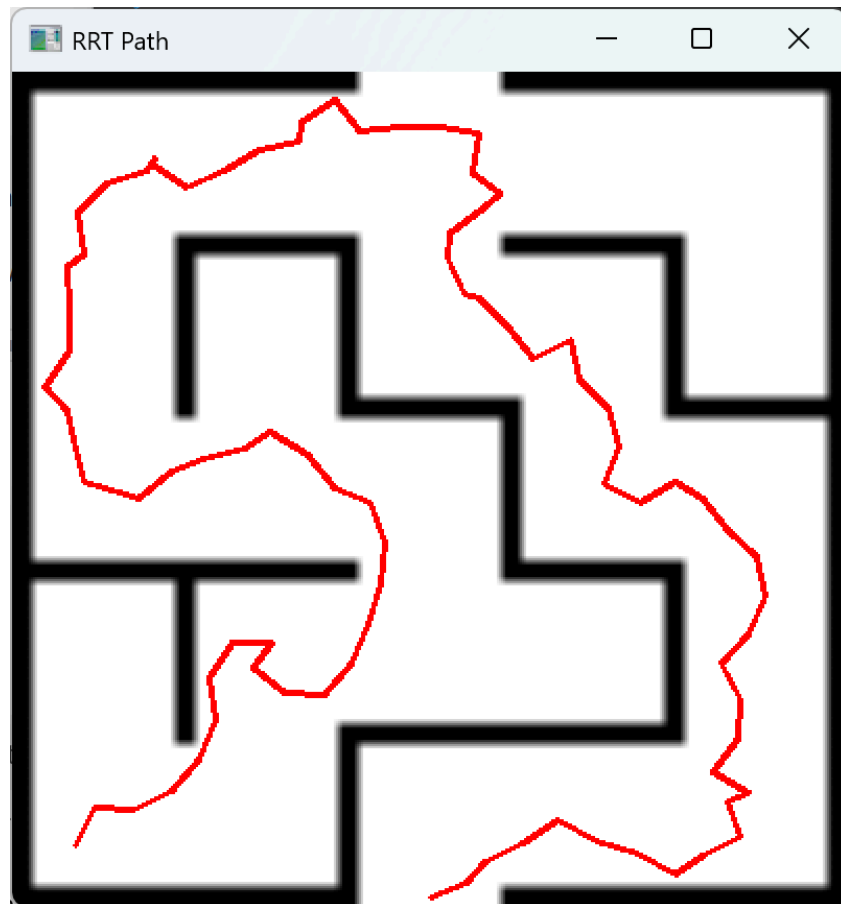
    new = steer(near, rand)
    if free(new) and not collision(near, new):
        nodes.append(new)
        parent[new] = near

    if dist(new, goal) < step and not collision(new, goal):
        parent[goal] = new
        found = True
        break

if found:
    cur = goal
    while parent[cur] is not None:
        p = parent[cur]
        cv2.line(img, p, cur, (0, 0, 255), 2)
        cur = p

cv2.imshow("RRT Path", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



Rapidly-exploring Random Trees Star (RRT*)

- Choose best parent instead of nearest parent.
- In RRT*, we also check the possibility assuming that THE NEWLY GENERATED NODE IS THE BEST PARENT OF ITS BEST PARENT.
- Eg, if nodes are A, B, C

$A \rightarrow B \rightarrow C$

where C is newly generated node and B is best parent

- We check what happens in the case where $A \rightarrow C \rightarrow B$
- If the new cost is less than old cost, we consider this case on the above line.

Task: Implement RRT* on the given maze.

```
import cv2
import numpy as np
import random
```

```

import math

img = cv2.imread(r"C:\Users\jasti\Downloads\maze.png")

img = cv2.resize(img, (400, 400))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, bw = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

start = (30, 370)
goal = (200, 395)

nodes = [start]
parent = {start: None}
cost = {start: 0}

step = 20
radius = 40

def free(p):
    return 0 <= p[0] < 400 and 0 <= p[1] < 400 and bw[p[1], p[0]] == 255

def dist(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

def steer(a, b):
    d = dist(a, b)
    if d < step:
        return b
    x = int(a[0] + step * (b[0]-a[0]) / d)
    y = int(a[1] + step * (b[1]-a[1]) / d)
    return (x, y)

def collision(a, b):
    for i in range(21):
        x = int(a[0] + (b[0]-a[0]) * i / 20)
        y = int(a[1] + (b[1]-a[1]) * i / 20)
        if bw[y, x] == 0:

```

```

        return True
    return False

for _ in range(6000):
    rand = (random.randint(0, 399), random.randint(0, 399))

    if not free(rand):
        continue

    nearest = nodes[0]
    for n in nodes:
        if dist(n, rand) < dist(nearest, rand):
            nearest = n

    new = steer(nearest, rand)

    if not free(new) or collision(nearest, new):
        continue

    best_parent = nearest
    best_cost = cost[nearest] + dist(nearest, new)

    for n in nodes:
        if dist(n, new) < radius and not collision(n, new):
            c = cost[n] + dist(n, new)
            if c < best_cost:
                best_cost = c
                best_parent = n

    nodes.append(new)
    parent[new] = best_parent
    cost[new] = best_cost

    for n in nodes:
        if n != new and dist(n, new) < radius and not collision(new, n):
            if cost[new] + dist(new, n) < cost[n]:
                parent[n] = new
                cost[n] = cost[new] + dist(new, n)

```

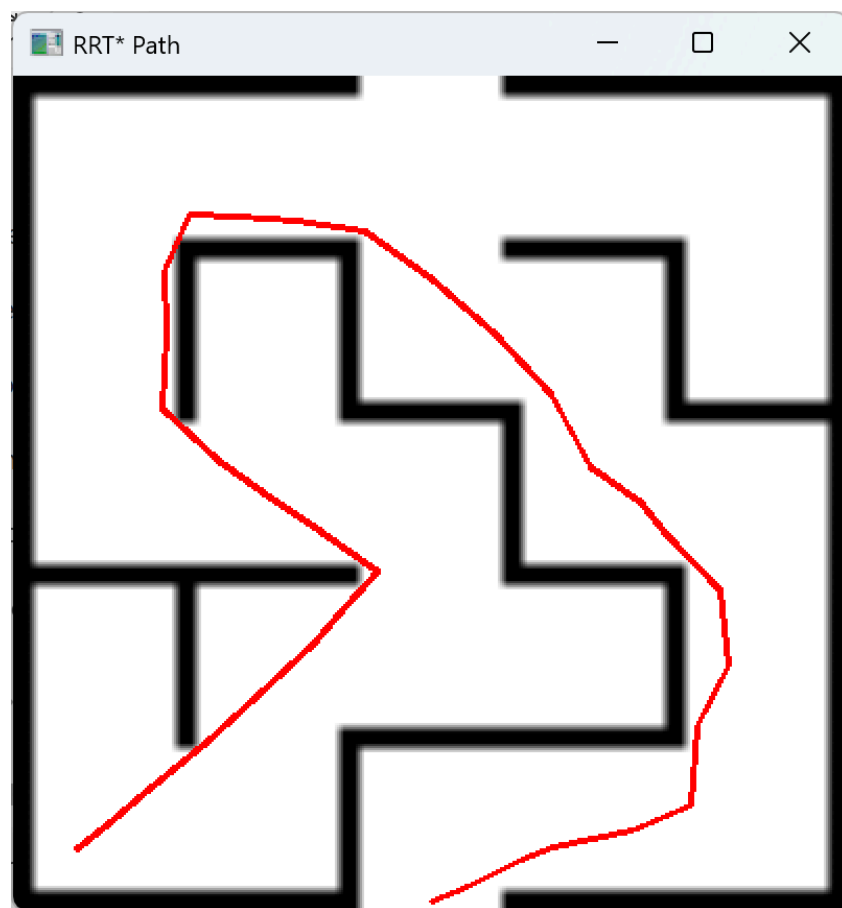
```

if dist(new, goal) < step and not collision(new, goal):
    parent[goal] = new
    cost[goal] = cost[new] + dist(new, goal)
    break

cur = goal
while cur in parent and parent[cur] is not None:
    p = parent[cur]
    cv2.line(img, p, cur, (0, 0, 255), 2)
    cur = p

cv2.imshow("RRT* Path", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



Final Day Project

Python Code

```
import cv2 as cv
import serial
import time
import numpy as np
import keyboard as kb
import math

coords = (x, y, z)

l1 = 9.2
l2 = 9.8

ser = serial.Serial(port='COM8', baudrate=9600, timeout=.1)

time.sleep(2)

def Publish(x, y, z):
    try:
        theta0 = math.atan2(y, x)
        x = math.hypot(x, y)
        r = math.hypot(x, z)

        
$$D = (r^2 - l1^2 - l2^2) / (2 * l1 * l2)$$


        if D > 1.0:
            D = 1.0
        elif D < -1.0:
            D = -1.0

        D = np.clip(D, -1, 1)

        dir = 1
        t = dir * math.sqrt(1 - D**2)
        theta2 = math.atan2(t, D)

        a = math.atan2(z, x)
        b = math.atan2(l2 * math.sin(theta2), l1 + l2 * math.cos(theta2))
```

```

theta1 = a - b

servo0 = np.clip(s0 + math.degrees(theta0), 0, 180)
servo1 = np.clip(s1 + math.degrees(theta1), 0, 180)
servo2 = np.clip(s2 + math.degrees(theta2), 0, 180)

packet = f"{servo0}, {servo1}, {servo2}\n"

ser.write(packet.encode('utf-8'))

except KeyboardInterrupt:
    ser.close()

wCam, hCam = 1200, 980

cap = cv.VideoCapture(1)
cFactor = 15.4 / 1780

cap.set(3, wCam)
cap.set(4, hCam)

pTime = 0

while True:
    ret, img = cap.read()
    if not ret:
        break

    cTime = time.time()
    fps = 1 / (cTime - pTime + 1e-8)
    pTime = cTime

    img = cv.flip(img, 1)

    cv.putText(
        img,
        f'FPS: {int(fps)}',

```

```

    (40, 70),
    cv.FONT_HERSHEY_COMPLEX,
    3,
    (255, 0, 255),
    3
)

hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)

lower = np.array([95, 60, 50])
upper = np.array([130, 255, 255])

mask = cv.inRange(hsv, lower, upper)

kernel = np.ones((5, 5), np.uint8)
mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)

ys, xs = np.where(mask > 0)

if len(xs) > 0 and len(ys) > 0:
    cx_pix = xs.mean()
    cy_pix = ys.mean()

    cx = cx_pix * cFactor * 2
    cy = cy_pix * cFactor * 2

    xLen = len(xs)

    k = 1
    c = 0

    if xLen != 0:
        cz = math.sqrt(k * 1080000 / (xLen - 1080000 * c)) * 2
    else:
        cz = cy

    print(cx, cy, cz)
else:

```

```

        print("no ball detected")

cv.imshow("Img", img)
cv.imshow("img2", mask)

if cv.waitKey(20) & 0xFF == 27:
    break

if kb.is_pressed('ctrl'):
    break

cap.release()
cv.destroyAllWindows()

```

Arduino Code

```

char receivedData[64];
char* token_abc;

void setup() {
    Serial.begin(9600);
}

void loop() {
    if (Serial.available()) {
        int bytesRead = Serial.readBytesUntil('\n', receivedData, sizeof(receivedData) - 1);

        receivedData[bytesRead] = '\0';

        token_abc = strtok(receivedData, ",");

        String token = String(token_abc);
        int a = token.toInt();

        int b = (String(strtok(NULL, ",")).toInt());

        int c = (String(strtok(NULL, ",")).toInt());
    }
}

```

```
}  
}
```

Python to Serial Monitor Code

```
import serial  
import time  
  
ser = serial.Serial('COM12', 9600, timeout=1)  
  
time.sleep(2)  
  
try:  
    while True:  
        ser.write(("2,4,8" + '\n').encode('utf-8'))  
  
except KeyboardInterrupt:  
    ser.close()
```

Declaration

This report is an original work submitted by **Varchas Jasti** as part of the requirements related to the aforementioned event. The content presented reflects personal observations, analysis, and learning outcomes derived from the event. This document is submitted in good faith and is intended solely for academic and evaluative purposes.